



ADVANCED TERRAFORM & NGINX MULTI-TIER ARCHITECTURE

Instructor: Waqas Saleem



DECEMBER 29, 2025

NAME: AMINA NOORREG NO. 2023-BSE-007

Table of Contents

1. Executive Summary	2
2. Architecture Design	3
2.1 Architecture Overview	3
2.2 Architecture Diagram	3
2.3 Component Descriptions	3
2.4 Network Topology	4
2.5 Security Design	4
3. Implementation Details	5
3.1 Infrastructure Setup.....	5
3.2 Webserver Module.....	25
3.3 Server Configuration Scripts	32
3.4 Infrastructure Deployment	41
3.5 Nginx Configuration & Testing	47
3.6 Documentation & Cleanup	58
4. Testing Results	63
4.1 Load Balancing Tests	63
4.2 Cache Performance Tests	64
4.3 High Availability Tests	64
4.4 Security Tests	65
4.5 Performance Metrics	66
5. Challenges & Solutions	68
5.1 Infrastructure Provisioning Challenges	68
5.2 SSH Access and Key Management Issues	68
5.3 Nginx Backend Connectivity Problems	68
5.4 SSL Certificate and HTTPS Configuration Challenges	69
5.5 Cache and Performance Tuning Issues	69
5.6 High Availability and Failover Testing Challenges	69
6. Conclusion	71
6.1 Summary of Work Completed	71
6.2 Skills Acquired	71
6.3 Future Improvements	71
7. Appendices	72
7.1 Appendix A: References	72

Assignment No# 2

1. Executive Summary

Overview of the Assignment

This assignment focuses on designing, implementing, and deploying a multi-tier web infrastructure on AWS using Terraform as Infrastructure as Code (IaC). The project automates the provisioning of networking, security, compute resources, and server configurations to host a scalable and secure web application.

Infrastructure Deployed

The infrastructure consists of:

- A custom VPC with public subnet
- Internet Gateway and routing configuration
- Security Groups for controlled access
- One Nginx reverse proxy server
- Three Apache backend web servers (two primary, one backup)
- Automated server configuration using shell scripts
- Load balancing, caching, and SSL termination via Nginx

Key Achievements

- Successfully implemented modular Terraform architecture
- Automated EC2 provisioning with reusable modules
- Configured secure networking and access controls
- Implemented HTTPS, caching, and load balancing using Nginx
- Verified high availability with backup server failover
- Completed deployment, testing, and cleanup without manual AWS resource creation

2. Architecture Design

2.1 Architecture Overview

The system follows a three-tier architecture:

- Presentation Layer: Nginx reverse proxy
- Application Layer: Apache backend servers
- Infrastructure Layer: AWS networking and security resources

The architecture ensures scalability, security, and high availability.

2.2 Architecture Diagram

(Insert diagram or ASCII diagram here)

Internet

|

HTTPS / HTTP

|

[Nginx Reverse Proxy]

|

| | |

[Web-1] [Web-2] [Web-3]

Primary Primary Backup

2.3 Component Descriptions

Virtual Private Cloud (VPC)

A custom VPC was created using Terraform to isolate resources and provide full control over IP addressing.

Subnet

A public subnet allows EC2 instances to receive public IPs and communicate with the internet.

EC2 Instances

- Nginx Server: Acts as a load balancer, SSL terminator, and cache server

- Apache Servers: Serve dynamic web content and respond to Nginx requests

Terraform Modules

- Networking Module: VPC, subnet, route table, internet gateway
- Security Module: Security groups for Nginx and backend servers
- Webserver Module: EC2 instance creation with user-data scripts

2.4 Network Topology

- Internet Gateway connects VPC to the internet
- Route table directs outbound traffic
- Public subnet hosts all EC2 instances
- Backend servers accept traffic only from Nginx security group

2.5 Security Design

- SSH access restricted to user's public IP
- HTTP/HTTPS allowed only where required
- Backend servers protected from direct internet access
- HTTPS enforced using SSL certificates
- Security headers added for enhanced protection

3. Implementation Details

3.1: Infrastructure Setup

3.1.1 Project Structure

STEP 1: Open Command Prompt

1. Press **Windows + R**
2. Type cmd
3. Press **Enter** cd Desktop

STEP 2: Create Main Project Folder

```
mkdir Assignment2
```

```
cd Assignment2
```

STEP 3: Create Main Terraform Files

```
type nul > main.tf
```

```
type nul > variables.tf
```

```
type nul > outputs.tf
```

```
type nul > locals.tf
```

```
type nul > terraform.tfvars
```

```
type nul > .gitignore
```

```
type nul > README.md
```

STEP 4: Create Modules & Scripts Folders

```
mkdir modules
```

```
mkdir scripts
```

STEP 5: Create Module Subfolders

```
mkdir modules\networking
```

```
mkdir modules\security
```

```
mkdir modules\webserver
```

STEP 6: Create Files Inside Modules

Networking module

```
type nul > modules\networking\main.tf
```

```
type nul > modules\networking\variables.tf
```

```
type nul > modules\networking\outputs.tf
```

Security module

```
type nul > modules\security\main.tf
```

```
type nul > modules\security\variables.tf
```

```
type nul > modules\security\outputs.tf
```

Webserver module

```
type nul > modules\webserver\main.tf
```

```
type nul > modules\webserver\variables.tf
```

```
type nul > modules\webserver\outputs.tf
```

STEP 7: Create Script Files

```
type nul > scripts\nginx-setup.sh
```

```
type nul > scripts\apache-setup.sh
```

STEP 8: Verify Folder Structure

Run:

```
tree /F
```

```
C:\Users\Lenovo\Desktop\Assignment2>tree /F
Folder PATH listing
Volume serial number is 10F6-E847
C:..
  .gitignore
  locals.tf
  main.tf
  outputs.tf
  README.md
  terraform.tfvars
  variables.tf
  modules
    networking
      main.tf
      outputs.tf
      variables.tf
    security
      main.tf
      outputs.tf
      variables.tf
    webserver
      main.tf
      outputs.tf
      variables.tf
  scripts
    apache-setup.sh
    nginx-setup.sh
```

STEP 9: Add Content to .gitignore

```

.gitignore - Notepad
File Edit Format View Help

# Terraform working directory
.terraform/
.terraform.lock.hcl

# Sensitive variable files
terraform.tfvars

# Logs
crash.log

# SSH keys
*.pem
*.key

# Editor folders
.vscode/
.idea/

# OS files
Thumbs.db
.DS_Store

```

3.1.2 Variable Configuration

STEP 1: variables.tf

Open file in CMD:

notepad variables.tf

Paste **everything below**:

```

#####

# Networking Variables

#####

variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type        = string
  validation {
    condition = can(cidrnetmask(var.vpc_cidr_block))
    error_message = "The VPC CIDR block must be a valid IPv4 CIDR range."
  }
}

variable "subnet_cidr_block" {
  description = "CIDR block for the public subnet"
  type        = string
  validation {

```



```

    condition    = can(cidrnetmask(var.subnet_cidr_block))

    error_message = "The subnet CIDR block must be a valid IPv4 CIDR range."
}

}

variable "availability_zone" {

    description = "Availability Zone where resources will be deployed"

    type    = string
}

#####

# Environment Variables

#####

variable "env_prefix" {

    description = "Environment name prefix (e.g., dev, staging, prod)"

    type    = string

    default = "dev"

}

#####

# EC2 Variables

#####

variable "instance_type" {

    description = "EC2 instance type for web servers"

    type    = string

    default = "t3.micro"

}

variable "public_key" {

    description = "Path to the SSH public key"

    type    = string

}

variable "private_key" {

    description = "Path to the SSH private key"

    type    = string

    sensitive = true

}

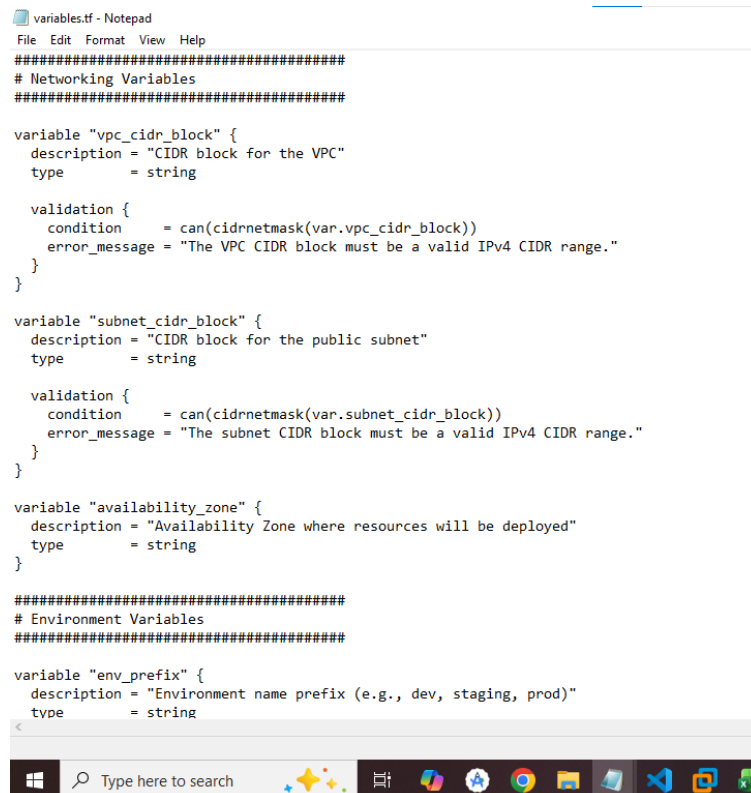
```

```
#####
```

```
# Backend Servers Configuration
```

```
#####
```

```
variable "backend_servers" {  
  
  description = "List of backend web servers with name and setup script"  
  
  type = list(object({  
  
    name      = string  
  
    script_path = string  
  
  }))  
}
```



```
variables.tf - Notepad  
File Edit Format View Help  
#####  
# Networking Variables  
#####  
  
variable "vpc_cidr_block" {  
  description = "CIDR block for the VPC"  
  type        = string  
  
  validation {  
    condition = can(cidrnetmask(var.vpc_cidr_block))  
    error_message = "The VPC CIDR block must be a valid IPv4 CIDR range."  
  }  
}  
  
variable "subnet_cidr_block" {  
  description = "CIDR block for the public subnet"  
  type        = string  
  
  validation {  
    condition = can(cidrnetmask(var.subnet_cidr_block))  
    error_message = "The subnet CIDR block must be a valid IPv4 CIDR range."  
  }  
}  
  
variable "availability_zone" {  
  description = "Availability Zone where resources will be deployed"  
  type        = string  
}  
  
#####  
# Environment Variables  
#####  
  
variable "env_prefix" {  
  description = "Environment name prefix (e.g., dev, staging, prod)"  
  type        = string
```

STEP 2: terraform.tfvars

Open file:

notepad terraform.tfvars

Paste:

```
vpc_cidr_block = "10.0.0.0/16"
```

```
subnet_cidr_block = "10.0.10.0/24"
```

```
availability_zone = "me-central-1a"
```

```
env_prefix = "prod"
```

```

instance_type = "t3.micro"

public_key = "~/ssh/id_ed25519.pub"
private_key = "~/ssh/id_ed25519"

backend_servers = [
  {
    name      = "web-1"
    script_path = "scripts/apache-setup.sh"
  },
  {
    name      = "web-2"
    script_path = "scripts/apache-setup.sh"
  },
  {
    name      = "web-3"
    script_path = "scripts/apache-setup.sh"
  }
]

```



```

File Edit Format View Help
vpc_cidr_block      = "10.0.0/16"
subnet_cidr_block  = "10.0.10.0/24"
availability_zone   = "me-central-1a"

env_prefix          = "prod"
instance_type       = "t3.micro"

public_key          = "~/ssh/id_ed25519.pub"
private_key         = "~/ssh/id_ed25519"

backend_servers = [
  {
    name      = "web-1"
    script_path = "scripts/apache-setup.sh"
  },
  {
    name      = "web-2"
    script_path = "scripts/apache-setup.sh"
  },
  {
    name      = "web-3"
    script_path = "scripts/apache-setup.sh"
  }
]

```

3.1.3 Networking Module

You will only edit files inside:

modules/networking/

STEP 1: modules/networking/variables.tf

Open in CMD:

notepad modules\networking\variables.tf

Paste:

```
variable "vpc_cidr_block" {  
    description = "CIDR block for the VPC"  
    type      = string  
}  
  
variable "subnet_cidr_block" {  
    description = "CIDR block for the public subnet"  
    type      = string  
}  
  
variable "availability_zone" {  
    description = "Availability zone for the subnet"  
    type      = string  
}  
  
variable "env_prefix" {  
    description = "Environment prefix for resource tagging"  
    type      = string  
}
```

Save and close.

STEP 2: modules/networking/main.tf

Open:

notepad modules\networking\main.tf

Paste **everything below**:

```
#####  
# VPC  
#####  
  
resource "aws_vpc" "this" {  
    cidr_block      = var.vpc_cidr_block  
    enable_dns_support = true  
    enable_dns_hostnames = true
```

```

tags = {
    Name = "${var.env_prefix}-vpc"
}
}

#####

# Public Subnet

#####

resource "aws_subnet" "this" {
    vpc_id      = aws_vpc.this.id
    cidr_block  = var.subnet_cidr_block
    availability_zone = var.availability_zone
    map_public_ip_on_launch = true
    tags = {
        Name = "${var.env_prefix}-public-subnet"
    }
}

#####

# Internet Gateway

#####

resource "aws_internet_gateway" "this" {
    vpc_id = aws_vpc.this.id
    tags = {
        Name = "${var.env_prefix}-igw"
    }
}

#####

# Route Table

#####

resource "aws_route_table" "this" {
    vpc_id = aws_vpc.this.id
    route {
        cidr_block = "0.0.0.0/0"
        gateway_id = aws_internet_gateway.this.id
    }
}

```

```

}

tags = {
    Name = "${var.env_prefix}-public-rt"
}
}

#####

# Route Table Association

#####

resource "aws_route_table_association" "this" {

    subnet_id    = aws_subnet.this.id

    route_table_id = aws_route_table.this.id
}

```



```

main.tf - Notepad
File Edit Format View Help
#####
# VPC
#####

resource "aws_vpc" "this" {
    cidr_block      = var.vpc_cidr_block
    enable_dns_support = true
    enable_dns_hostnames = true

    tags = {
        Name = "${var.env_prefix}-vpc"
    }
}

#####
# Public Subnet
#####

resource "aws_subnet" "this" {
    vpc_id            = aws_vpc.this.id
    cidr_block        = var.subnet_cidr_block
    availability_zone  = var.availability_zone
    map_public_ip_on_launch = true

    tags = {
        Name = "${var.env_prefix}-public-subnet"
    }
}

#####
# Internet Gateway
#####

resource "aws_internet_gateway" "this" {
    vpc_id = aws_vpc.this.id
}

```

STEP 3: modules/networking/outputs.tf

Open:

notepad modules\networking\outputs.tf

Paste:

```

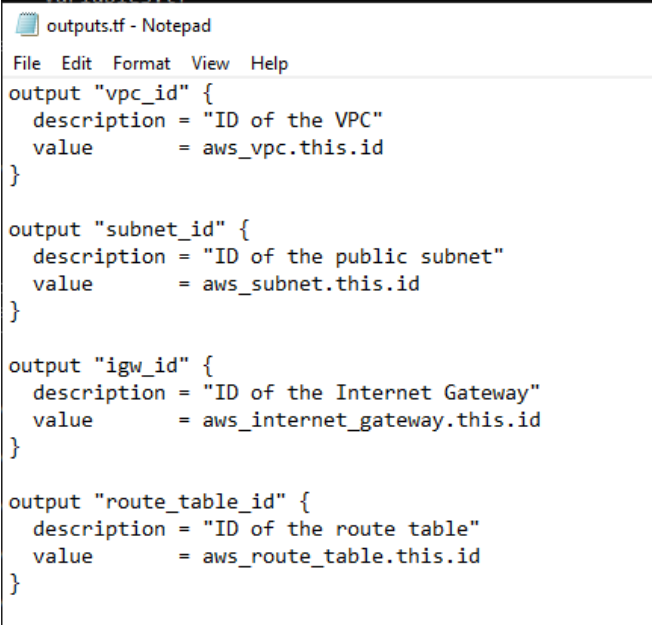
output "vpc_id" {
    description = "ID of the VPC"
    value      = aws_vpc.this.id
}

output "subnet_id" {
    description = "ID of the public subnet"
    value      = aws_subnet.this.id
}

output "igw_id" {
    description = "ID of the Internet Gateway"
    value      = aws_internet_gateway.this.id
}

output "route_table_id" {
    description = "ID of the route table"
    value      = aws_route_table.this.id
}

```



The screenshot shows a Notepad window titled 'outputs.tf - Notepad'. The menu bar includes 'File', 'Edit', 'Format', 'View', and 'Help'. The text content of the file is the same Terraform code block shown above, defining four output variables: vpc_id, subnet_id, igw_id, and route_table_id, each with a description and a value reference to an AWS resource.

3.1.4 Security Module

You will work **only inside**:

modules/security/

STEP 1: modules/security/variables.tf

Open with CMD:

notepad modules\security\variables.tf

Paste:

```
variable "vpc_id" {
    description = "VPC ID where security groups will be created"
    type      = string
}

variable "env_prefix" {
    description = "Environment prefix for resource naming and tagging"
    type      = string
}

variable "my_ip" {
    description = "Your public IP address in CIDR format (e.g., 203.0.113.5/32)"
    type      = string
}
```

Save and close.

STEP 2: modules/security/main.tf (CORE FILE)

Open:

notepad modules\security\main.tf

Paste **everything below**:

```
#####

# Nginx Security Group

#####

resource "aws_security_group" "nginx_sg" {
    name     = "${var.env_prefix}-nginx-sg"
    description = "Security group for Nginx reverse proxy"
    vpc_id   = var.vpc_id

    ingress {
        description = "SSH from my IP"
        from_port   = 22
        to_port     = 22
        protocol    = "tcp"
        cidr_blocks = [var.my_ip]
    }
}
```



```

ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}

ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}

egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}

tags = {
    Name = "${var.env_prefix}-nginx-sg"
}

}

#####

# Backend Security Group

#####

resource "aws_security_group" "backend_sg" {
    name           = "${var.env_prefix}-backend-sg"
    description    = "Security group for backend web servers"
    vpc_id         = var.vpc_id
    ingress {

```

```

description = "SSH from my IP"

from_port = 22

to_port = 22

protocol = "tcp"

cidr_blocks = [var.my_ip]
}

ingress {

description = "HTTP from Nginx security group"

from_port = 80

to_port = 80

protocol = "tcp"

security_groups = [aws_security_group.nginx_sg.id]
}

egress {

description = "Allow all outbound traffic"

from_port = 0

to_port = 0

protocol = "-1"

cidr_blocks = ["0.0.0.0/0"]
}

tags = {

Name = "${var.env_prefix}-backend-sg"
}
}

```

```

main.tf - Notepad
File Edit Format View Help
#####
# Nginx Security Group
#####

resource "aws_security_group" "nginx_sg" {
  name           = "${var.env_prefix}-nginx-sg"
  description    = "Security group for Nginx reverse proxy"
  vpc_id        = var.vpc_id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
  }
}

```

STEP 3: modules/security/outputs.tf

Open:

notepad modules\security\outputs.tf

Paste:

```

output "nginx_sg_id" {
  description = "ID of the Nginx security group"
  value       = aws_security_group.nginx_sg.id
}

output "backend_sg_id" {
  description = "ID of the backend security group"
  value       = aws_security_group.backend_sg.id
}

```

Save and close.

STEP 4: AWS Console Screenshot (After Apply)

After you later run:

terraform apply

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo>terraform -version
Terraform v1.14.3
on windows_amd64

C:\Users\Lenovo>terraform apply

Error: No configuration files

Apply requires configuration to be present. Applying without a configuration would mark everything for destruction,
which is normally not what is desired. If you would like to destroy everything, run 'terraform destroy' instead.

C:\Users\Lenovo>terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.

C:\Users\Lenovo>terraform plan

Error: No configuration files

Plan requires configuration to be present. Planning without a configuration would mark everything for destruction,
which is normally not what is desired. If you would like to destroy everything, run plan with the -destroy option.
Otherwise, create a Terraform configuration file (.tf file) and try again.

C:\Users\Lenovo>terraform apply

Error: No configuration files

Apply requires configuration to be present. Applying without a configuration would mark everything for destruction,
which is normally not what is desired. If you would like to destroy everything, run 'terraform destroy' instead.

C:\Users\Lenovo>terraform init
Terraform initialized in an empty directory!
```

3.1.4.1 Correct Way to Use Terraform (STEP-BY-STEP)

You must:

Create a **project folder**

Create .tf files

Run Terraform **inside that folder**

STEP 1: Create a Terraform Project Folder

Run this:

```
mkdir terraform-aws
```

```
cd terraform-aws
```

STEP 2: Create Your FIRST Terraform File

Create a file named:

```
main.tf
```

Easiest way (Notepad method):

1. Right-click → **New** → **Text Document**
2. Rename to:

```
main.tf
```

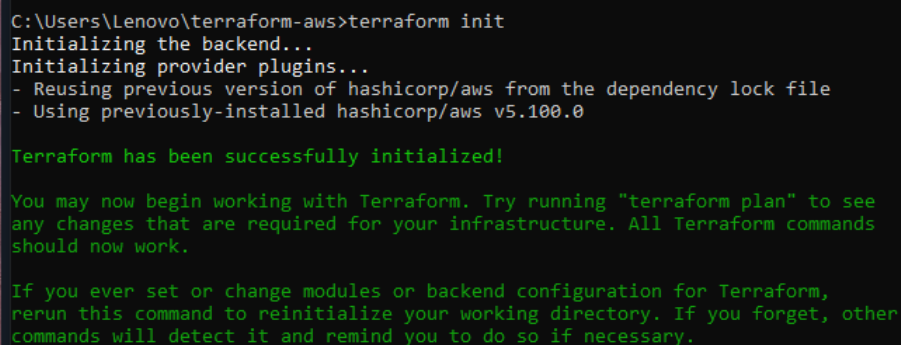
STEP 3: Add BASIC AWS Terraform Code (TEST)

Paste this into **main.tf**:

```
terraform {  
  required_version = ">= 1.5"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "me-central-1"  
}  
  
resource "aws_s3_bucket" "test_bucket" {  
  bucket = "amina-terraform-test-12345"  
}
```

STEP 4: Initialize Terraform

terraform init

A terminal window with a black background and white text. The output of the 'terraform init' command is shown. It starts with the path 'C:\Users\Lenovo\terraform-aws>', followed by 'terraform init'. The output includes 'Initializing the backend...', 'Initializing provider plugins...', and two bullet points: '- Reusing previous version of hashicorp/aws from the dependency lock file' and '- Using previously-installed hashicorp/aws v5.100.0'. A green line indicates 'Terraform has been successfully initialized!'. Below this, a green message says 'You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.' At the bottom, another green message states 'If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.'

STEP 5: Plan Infrastructure

terraform plan

```

C:\Users\Lenovo\terraform-aws>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.test_bucket will be created
+ resource "aws_s3_bucket" "test_bucket" {
+   acceleration_status      = (known after apply)
+   acl                     = (known after apply)
+   arn                    = (known after apply)
+   bucket                 = "amina-terraform-test-12345"
+   bucket_domain_name      = (known after apply)
+   bucket_prefix          = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy          = false
+   hosted_zone_id         = (known after apply)
+   id                    = (known after apply)
+   object_lock_enabled     = (known after apply)
+   policy                 = (known after apply)
+   region                = (known after apply)
+   request_payer          = (known after apply)
+   tags_all              = (known after apply)
+   website_domain         = (known after apply)
+   website_endpoint       = (known after apply)

+   cors_rule (known after apply)

+   grant (known after apply)

+   lifecycle_rule (known after apply)

+   logging (known after apply)

+   object_lock_configuration (known after apply)

+   replication_configuration (known after apply)

+   server_side_encryption_configuration (known after apply)

+   versioning (known after apply)

```

STEP 6: Apply Infrastructure

terraform apply

```

C:\Users\Lenovo\terraform-aws>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.test_bucket will be created
+ resource "aws_s3_bucket" "test_bucket" {
+   acceleration_status      = (known after apply)
+   acl                     = (known after apply)
+   arn                    = (known after apply)
+   bucket                 = "amina-terraform-test-12345"
+   bucket_domain_name      = (known after apply)
+   bucket_prefix          = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy          = false
+   hosted_zone_id         = (known after apply)
+   id                    = (known after apply)
+   object_lock_enabled     = (known after apply)
+   policy                 = (known after apply)
+   region                = (known after apply)
+   request_payer          = (known after apply)
+   tags_all              = (known after apply)
+   website_domain         = (known after apply)
+   website_endpoint       = (known after apply)

+   cors_rule (known after apply)

+   grant (known after apply)

+   lifecycle_rule (known after apply)

+   logging (known after apply)

+   object_lock_configuration (known after apply)

+   replication_configuration (known after apply)

+   server_side_encryption_configuration (known after apply)

+   versioning (known after apply)

```

Type:

Yes

```

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.test_bucket: Creating...
aws_s3_bucket.test_bucket: Creation complete after 2s [id=amina-terraform-test-12345]

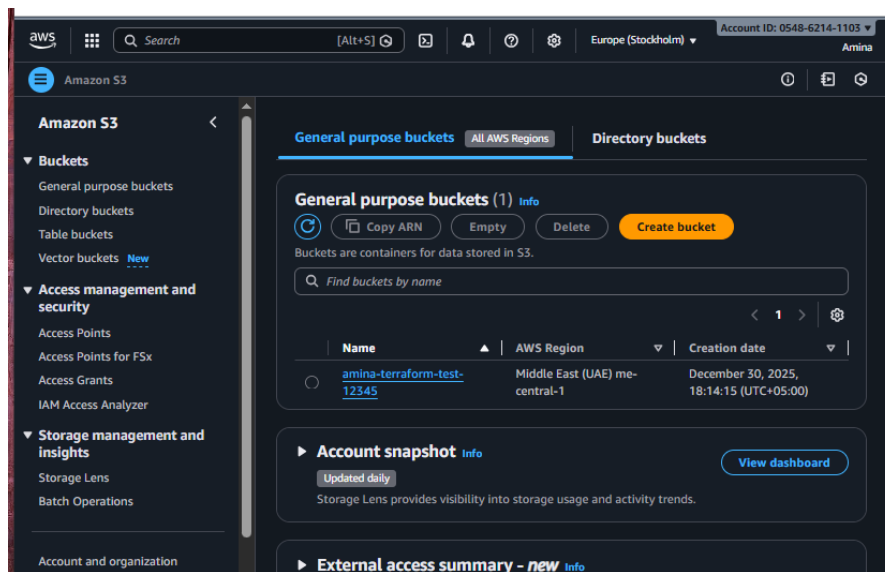
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

C:\Users\Lenovo\terraform-aws>

```

STEP 7: Verify in AWS Console

- Go to S3
- You'll see your bucket



STEP 8: Clean Up (VERY IMPORTANT)

To avoid AWS charges:

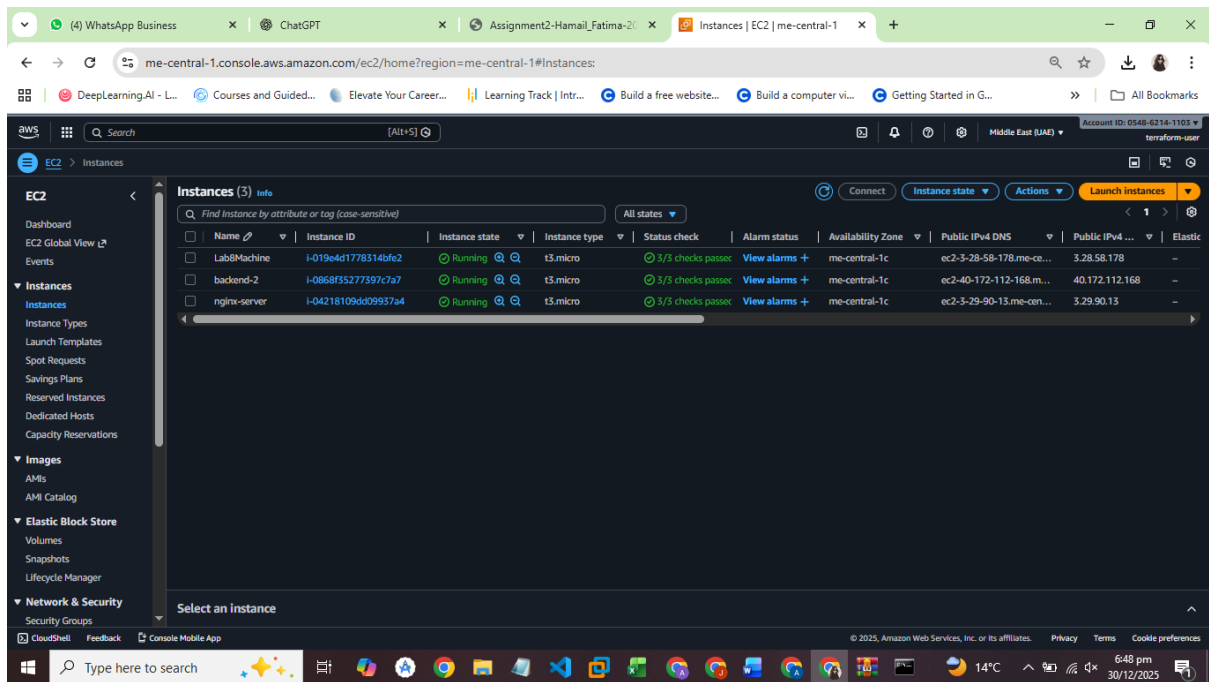
terraform destroy

Type:

yes

Go to **AWS Console** → **EC2** → **Security Groups** and take a screenshot showing:

- `${env_prefix}-nginx-sg`
- `${env_prefix}-backend-sg`
- Ingress & egress rules visible



3.1.5 Locals Configuration

STEP 1: locals.tf

Open CMD:

notepad locals.tf

Paste:

```
#####

# Dynamic IP Detection

#####

data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  # Get your current public IP dynamically
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  #####

  # Common Tags

  #####

  common_tags = {
    Environment = var.env_prefix

    Project    = "Assignment-2"
```



```

    ManagedBy = "Terraform"
}

#####

# Backend Server Configurations

#####

backend_servers = [

    {

        name      = "web-1"

        suffix    = "1"

        script_path = "./scripts/apache-setup.sh"

    },

    {

        name      = "web-2"

        suffix    = "2"

        script_path = "./scripts/apache-setup.sh"

    },

    {

        name      = "web-3"

        suffix    = "3"

        script_path = "./scripts/apache-setup.sh"

    }

]

#####

# Resource Naming Convention

#####

resource_names = {

    vpc      = "${var.env_prefix}-vpc"

    public_subnet = "${var.env_prefix}-public-subnet"

    nginx_sg   = "${var.env_prefix}-nginx-sg"

    backend_sg = "${var.env_prefix}-backend-sg"

    public_rt  = "${var.env_prefix}-public-rt"

}

}

```

```
locals.tf - Notepad
File Edit Format View Help
#####
# Dynamic IP Detection
#####
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

locals {
  # Get your current public IP dynamically
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  #####
  # Common Tags
  #####
  common_tags = {
    Environment = var.env_prefix
    Project     = "Assignment-2"
    ManagedBy   = "Terraform"
  }

  #####
  # Backend Server Configurations
  #####
  backend_servers = [
    {
      name       = "web-1"
      suffix     = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-2"
      suffix     = "2"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-3"

```

3.2: Webserver Module

3.2.1 Module Design

Step 1: Create variables.tf for Webserver Module

type nul > modules\webserver\variables.tf

notepad modules\webserver\variables.tf

Paste this code:

```
variable "env_prefix" {
  description = "Environment prefix for resource naming"
  type       = string
}

variable "instance_name" {
  description = "Name of the instance"
  type       = string
}

variable "instance_type" {
```

```

description = "EC2 instance type"

type      = string
}

variable "availability_zone" {
description = "Availability zone for the instance"

type      = string
}

variable "vpc_id" {
description = "VPC ID"

type      = string
}

variable "subnet_id" {
description = "Subnet ID"

type      = string
}

variable "security_group_id" {
description = "Security group ID to attach"

type      = string
}

variable "public_key" {
description = "Path to SSH public key"

type      = string
}

variable "script_path" {
description = "Path to user data script"

type      = string
}

variable "instance_suffix" {
description = "Suffix to make resources unique"

type      = string
}

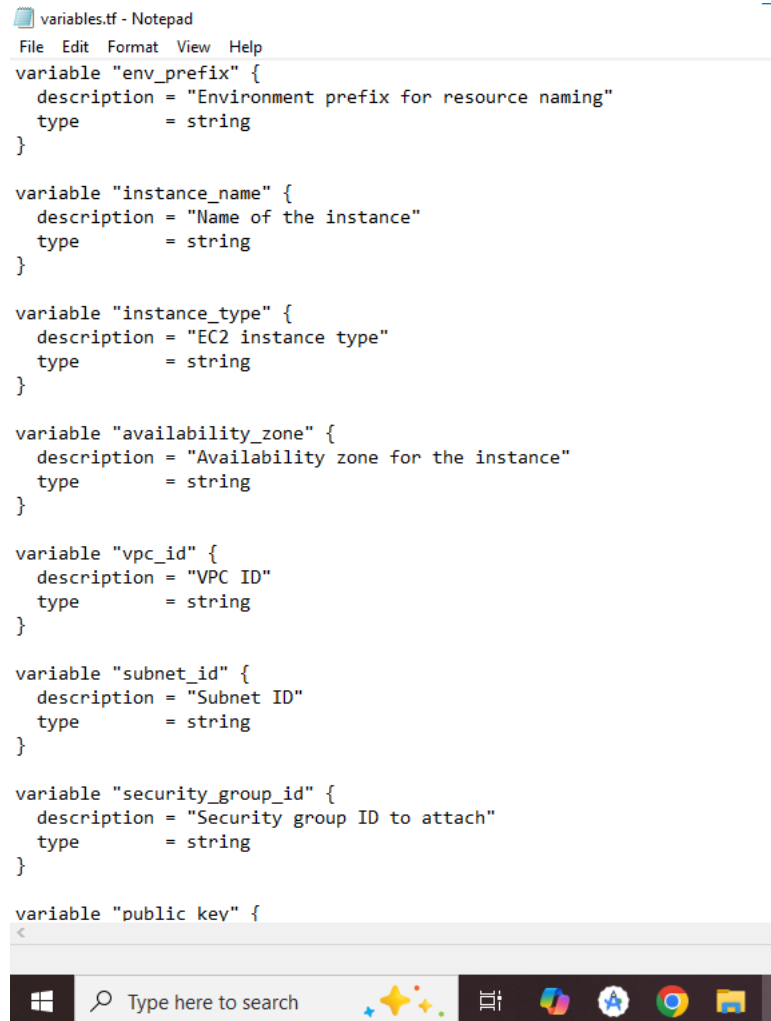
variable "common_tags" {
description = "Common tags map"

```

```

type    = map(string)
}

```



```

variables.tf - Notepad
File Edit Format View Help
variable "env_prefix" {
  description = "Environment prefix for resource naming"
  type        = string
}

variable "instance_name" {
  description = "Name of the instance"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "availability_zone" {
  description = "Availability zone for the instance"
  type        = string
}

variable "vpc_id" {
  description = "VPC ID"
  type        = string
}

variable "subnet_id" {
  description = "Subnet ID"
  type        = string
}

variable "security_group_id" {
  description = "Security group ID to attach"
  type        = string
}

variable "public_key" {

```

Step 2: Create main.tf for Webserver Module

type nul > modules\webserver/main.tf

notepad modules\webserver/main.tf

Paste this code:

```

#####
# AWS Key Pair
#####

resource "aws_key_pair" "this" {

  key_name  = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}-key"
  public_key = file(var.public_key)
}

#####

```

```
# EC2 Instance

#####

resource "aws_instance" "this" {

  ami          = "ami-0c9d7c0a2b0f5b123" # Amazon Linux 2023 (update if needed)

  instance_type = var.instance_type

  subnet_id     = var.subnet_id

  vpc_security_group_ids = [var.security_group_id]

  key_name      = aws_key_pair.this.key_name

  availability_zone = var.availability_zone

  associate_public_ip_address = true

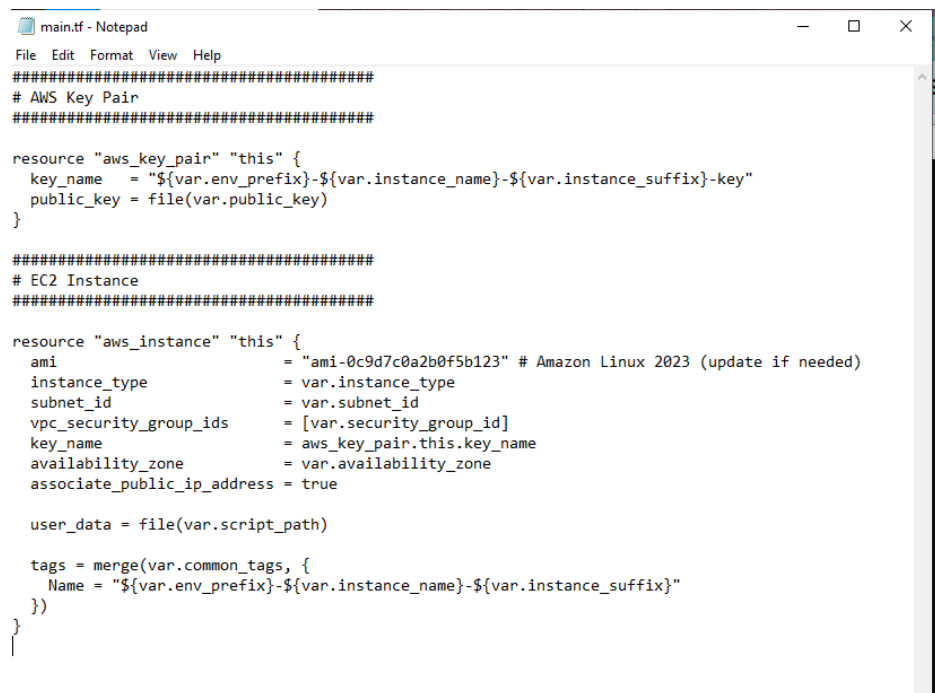
  user_data = file(var.script_path)

  tags = merge(var.common_tags, {

    Name = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"

  })

}
```



```
main.tf - Notepad
File Edit Format View Help
#####
# AWS Key Pair
#####

resource "aws_key_pair" "this" {
  key_name    = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}-key"
  public_key = file(var.public_key)
}

#####
# EC2 Instance
#####

resource "aws_instance" "this" {
  ami          = "ami-0c9d7c0a2b0f5b123" # Amazon Linux 2023 (update if needed)
  instance_type = var.instance_type
  subnet_id     = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  key_name      = aws_key_pair.this.key_name
  availability_zone = var.availability_zone
  associate_public_ip_address = true

  user_data = file(var.script_path)

  tags = merge(var.common_tags, {
    Name = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
  })
}
```

Step 3: Create outputs.tf

type nul > modules\webserver\outputs.tf

notepad modules\webserver\outputs.tf

Paste this code:

```
output "instance_id" {
```

```

description = "ID of the EC2 instance"

value      = aws_instance.this.id
}

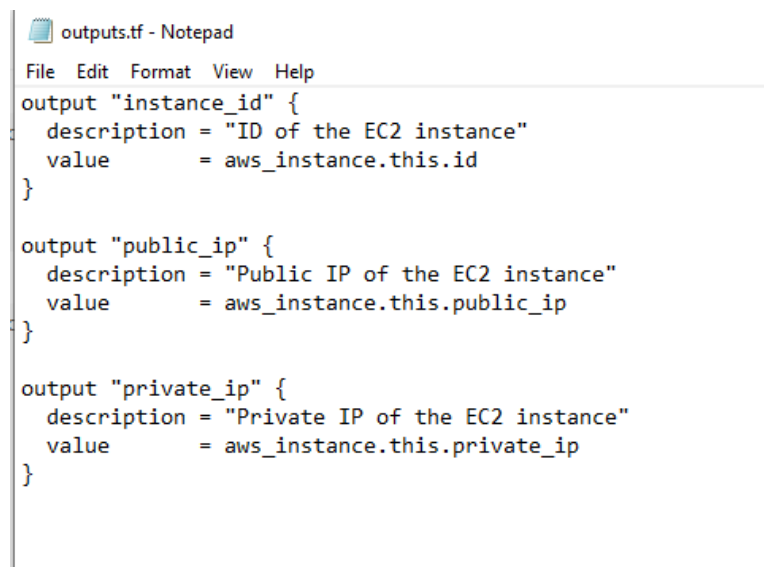
output "public_ip" {
description = "Public IP of the EC2 instance"

value      = aws_instance.this.public_ip
}

output "private_ip" {
description = "Private IP of the EC2 instance"

value      = aws_instance.this.private_ip
}

```



The screenshot shows a Notepad window titled "outputs.tf - Notepad". The window contains the following Terraform code:

```

output "instance_id" {
  description = "ID of the EC2 instance"
  value      = aws_instance.this.id
}

output "public_ip" {
  description = "Public IP of the EC2 instance"
  value      = aws_instance.this.public_ip
}

output "private_ip" {
  description = "Private IP of the EC2 instance"
  value      = aws_instance.this.private_ip
}

```

3.2.2 Module Usage

Step 1: Define Local Variables for Backend Servers

In locals.tf, define the backend servers you want to create dynamically:

```

locals {
  backend_servers = [
    {
      name      = "web-1"
      script_path = "./scripts/apache-setup.sh"
      suffix    = "web1"
    },
    {

```

```

    name      = "web-2"

    script_path = "./scripts/apache-setup.sh"

    suffix     = "web2"
  },
  {
    name      = "web-3"

    script_path = "./scripts/apache-setup.sh"

    suffix     = "web3"
  }
]

common_tags = {
  Project = "TerraformAssignment"
  Owner   = "Amina"
}
}

```

Step 2: Configure Root main.tf

Use the module syntax exactly as you wrote. Here's the corrected, ready-to-use version:

```

# Nginx Server

module "nginx_server" {

  source      = "./modules/webserver"

  env_prefix   = var.env_prefix

  instance_name = "nginx-proxy"

  instance_type = var.instance_type

  availability_zone = var.availability_zone

  vpc_id       = module.networking.vpc_id

  subnet_id    = module.networking.subnet_id

  security_group_id = module.security.nginx_sg_id

  public_key    = var.public_key

  script_path   = "./scripts/nginx-setup.sh"

  instance_suffix = "nginx"

  common_tags    = local.common_tags
}

# Backend Servers

```

```

module "backend_servers" {

  for_each = { for server in local.backend_servers : server.name => server }

  source      = "./modules/webserver"

  env_prefix   = var.env_prefix

  instance_name = each.value.name

  instance_type = var.instance_type

  availability_zone = var.availability_zone

  vpc_id       = module.networking.vpc_id

  subnet_id    = module.networking.subnet_id

  security_group_id = module.security.backend_sg_id

  public_key    = var.public_key

  script_path   = each.value.script_path

  instance_suffix = each.value.suffix

  common_tags   = local.common_tags

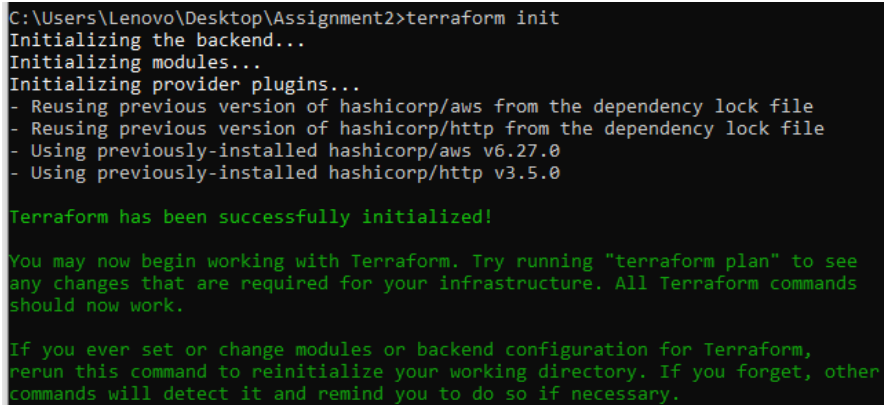
}

```

Step 3: Initialize Terraform

From your project root folder:

terraform init



```

C:\Users\Lenovo\Desktop\Assignment2>terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/http from the dependency lock file
- Using previously-installed hashicorp/aws v6.27.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

Step 5: Validate the Configuration

terraform validate

- Checks your .tf files for syntax errors.
- Should return: Success! The configuration is valid.

Step 6: Plan the Deployment

terraform plan -out=tfplan

- This shows what resources Terraform will create.

- It is **good practice to review** the plan before applying.
- Verify:
 - 1 Nginx server
 - 3 Backend servers
 - Correct instance types, scripts, and security groups

Step 7: Apply the Terraform Plan

terraform apply "tfplan"

- Confirm by typing yes when prompted.
- Terraform will now create your EC2 instances with the specified modules and scripts.

Step 8: Verify the Deployment

- Check AWS Console → EC2 → Instances
- You should see:
 - nginx-proxy server
 - web-1, web-2, web-3 servers
- Also, you can check from CLI:

aws ec2 describe-instances --filters "Name=tag:Name,Values=nginx-proxy,web-1,web-2,web-3"

Step 9: Required Final Screenshot



```

main.tf - Notepad
File Edit Format View Help
# Nginx Server (Reverse Proxy)
# =====
module "nginx_server" {
  source           = "./modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = "nginx"
  instance_type    = "t3.micro"
  availability_zone = "us-east-1a"
  vpc_id           = module.networking.vpc_id
  subnet_id        = module.networking.public_subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key       = file("~/ssh/id_rsa.pub")
  script_path      = "nginx-setup.sh"
  instance_suffix  = "1"
  common_tags      = var.common_tags
}
# =====
# Backend Servers (Apache)
# =====
module "backend_servers" {
  source = "./modules/webserver"
  for_each = toset(["1", "2", "3"])

  env_prefix       = var.env_prefix
  instance_name    = "web"
  instance_type    = "t3.micro"
  availability_zone = "us-east-1a"
  vpc_id           = module.networking.vpc_id
  subnet_id        = module.networking.public_subnet_id
  security_group_id = module.security.web_sg_id
  public_key       = file("~/ssh/id_rsa.pub")
  script_path      = "apache-setup.sh"
  instance_suffix  = each.key
  common_tags      = var.common_tags
}
  
```

3.3: Server Configuration Scripts

3.3.1 Apache Backend Server Script

Create scripts/apache-setup.sh that:

- Updates system packages
- Installs Apache HTTP Server
- Starts and enables Apache service
- Creates custom HTML page displaying:
 - Server name/hostname
 - Private IP address
 - Public IP address
 - Public DNS hostname
 - Custom message indicating which backend server (web-1, web-2, or web-3)
 - Timestamp of deployment
 - Server status (Primary/Backup)

Enhanced Script Requirements:

```
#!/bin/bash

set -e

# Update system
yum update -y

# Install Apache
yum install httpd -y

# Start and enable Apache
systemctl start httpd
systemctl enable httpd

# Get metadata token (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get instance metadata
PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/local-ipv4)

PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

PUBLIC_DNS=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
```

```

http://169.254.169.254/latest/meta-data/public-hostname)
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)

# Set hostname
hostnamectl set-hostname myapp-webserver

# Create custom HTML page
cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>Backend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
    }
    .container {
      background: rgba(255, 255, 255, 0.1);
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.37);
    }
    h1 { color: #fff; text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }
    .info { margin: 15px 0; padding: 10px; background: rgba(255,255,255,0.2); border-radius: 5px; }
    .label { font-weight: bold; color: #ffd700; }
  </style>
</head>
<body>
  <div class="container">
    <h1>🚀 Backend Web Server - Assignment 2</h1>
    <div class="info"><span class="label">Hostname:</span> ${hostname}</div>

```

```

<div class="info"><span class="label">Instance ID:</span> $INSTANCE_ID</div>

<div class="info"><span class="label">Private IP:</span> $PRIVATE_IP</div>

<div class="info"><span class="label">Public IP:</span> $PUBLIC_IP</div>

<div class="info"><span class="label">Public DNS:</span> $PUBLIC_DNS</div>

<div class="info"><span class="label">Deployed: </span> $(date)</div>

<div class="info"><span class="label">Status:</span> ☒ Active and Running</div>

<div class="info"><span class="label">Managed By:</span> Terraform</div>

```

```
</div>
```

```
</body>
```

```
</html>
```

EOF

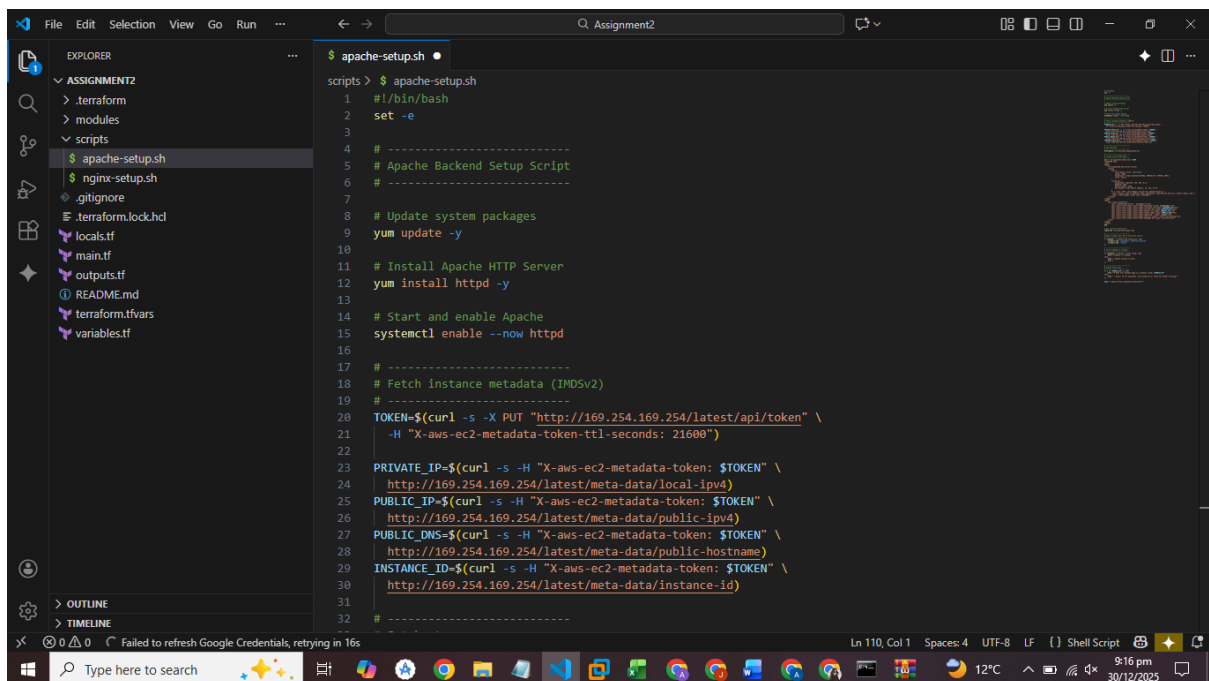
Set permissions

```
chmod 644 /var/www/html/index.html
```

```
echo "Apache setup completed successfully!"
```

Required Screenshots:

1.apache_script



```

$ apache-setup.sh
1  #!/bin/bash
2  set -e
3
4  # -----
5  # Apache Backend Setup Script
6  # -----
7
8  # Update system packages
9  yum update -y
10
11 # Install Apache HTTP Server
12 yum install httpd -y
13
14 # Start and enable Apache
15 systemctl enable --now httpd
16
17 # -----
18 # Fetch instance metadata (IMDSv2)
19 # -----
20 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
21 -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
22
23 PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
24 http://169.254.169.254/latest/meta-data/local-ipv4)
25 PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
26 http://169.254.169.254/latest/meta-data/public-ipv4)
27 PUBLIC_DNS=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
28 http://169.254.169.254/latest/meta-data/public-hostname)
29 INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
30 http://169.254.169.254/latest/meta-data/instance-id)
31
32 # -----

```

2. _backend_webpage



3.3.2 Nginx Server Setup Script

Create scripts/nginx-setup.sh that:

- Updates system packages
- Installs Nginx
- Generates self-signed SSL certificate
- Configures Nginx with:
 - HTTPS on port 443
 - HTTP to HTTPS redirect
 - Upstream backend servers (web-1, web-2, web-3)
 - Load balancing configuration
 - Caching configuration
 - Proper logging
 - Security headers

Enhanced Script Template:

```
#!/bin/bash
set -e

# Update and install Nginx
yum update -y
yum install -y nginx openssl

systemctl start nginx
systemctl enable nginx

# Create SSL directories
```

```

mkdir -p /etc/ssl/private

mkdir -p /etc/ssl/certs

# Get metadata token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth"

echo "Self-signed certificate created for IP: $PUBLIC_IP"

# Backup original config
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

# Create Nginx configuration

# Note: Backend IPs will be added manually after deployment
cat > /etc/nginx/nginx.conf <<'EOF'

user nginx;

worker_processes auto;

error_log /var/log/nginx/error.log notice;

pid /run/nginx. pid;


events {
    worker_connections 1024;
}

http {
    # Logging

```

```

log_format main '$remote_addr - $remote_user [$time_local] "$request" '
    '$status $body_bytes_sent "$http_referer" '
    '"$http_user_agent" "$http_x_forwarded_for" '
    'Cache: $upstream_cache_status';
access_log /var/log/nginx/access.log main;

# Basic settings

sendfile on;

tcp_nopush on;

keepalive_timeout 65;

types_hash_max_size 4096;

include /etc/nginx/mime.types;

default_type application/octet-stream;

# Gzip compression

gzip on;

gzip_vary on;

gzip_types text/plain text/css application/json application/javascript text/xml application/xml;

# Cache configuration

proxy_cache_path /var/cache/nginx
    levels=1:2
    keys_zone=my_cache:10m
    max_size=1g
    inactive=60m
    use_temp_path=off;

# Upstream backend servers

# PLACEHOLDER: Update these IPs after deployment

upstream backend_servers {
    # Primary servers (active load balancing)

    server BACKEND_IP_1:80;

    server BACKEND_IP_2:80;

    # Backup server (only used when primary servers are down)

    server BACKEND_IP_3:80 backup;
}

```

```

# HTTPS Server

server {
    listen 443 ssl http2;

    server_name _;

    # SSL Configuration

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Security Headers

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Proxy settings

    location / {
        proxy_pass http://backend_servers;

        # Proxy headers

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Cache settings

        proxy_cache my_cache;
        proxy_cache_valid 200 60m;
        proxy_cache_valid 404 10m;
        proxy_cache_key "$scheme$request_method$host$request_uri";
        proxy_cache_bypass $http_cache_control;
        add_header X-Cache-Status $upstream_cache_status;

        # Timeouts

        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
    }
}

```



```

        proxy_read_timeout 60s;
    }
    # Health check endpoint
    location /health {
        access_log off;

        return 200 "Nginx is healthy\n";

        add_header Content-Type text/plain;
    }
}

# HTTP Server (redirect to HTTPS)
server {
    listen 80;

    server_name _;

    location / {
        return 301 https://$host$request_uri;
    }

    # Allow health checks over HTTP
    location /health {
        access_log off;

        return 200 "Nginx is healthy\n";

        add_header Content-Type text/plain;
    }
}

}

EOF

# Create cache directory
mkdir -p /var/cache/nginx

chown -R nginx:nginx /var/cache/nginx

# Test and restart Nginx
nginx -t && systemctl restart nginx

echo "Nginx setup completed successfully!"

echo "Remember to update backend server IPs in /etc/nginx/nginx.conf"

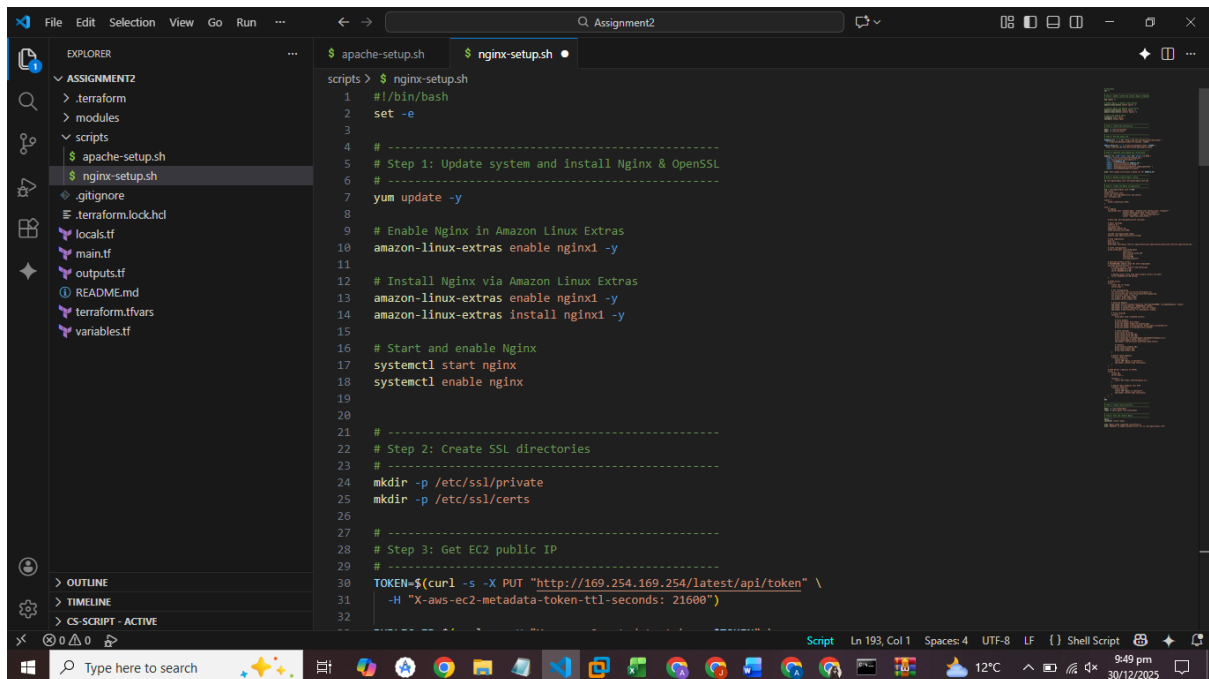
```

Tasks:

- Create script with SSL certificate generation
- Configure upstream with placeholder IPs
- Implement caching
- Add security headers
- Configure HTTP to HTTPS redirect
- Add health check endpoint

Required Screenshots:

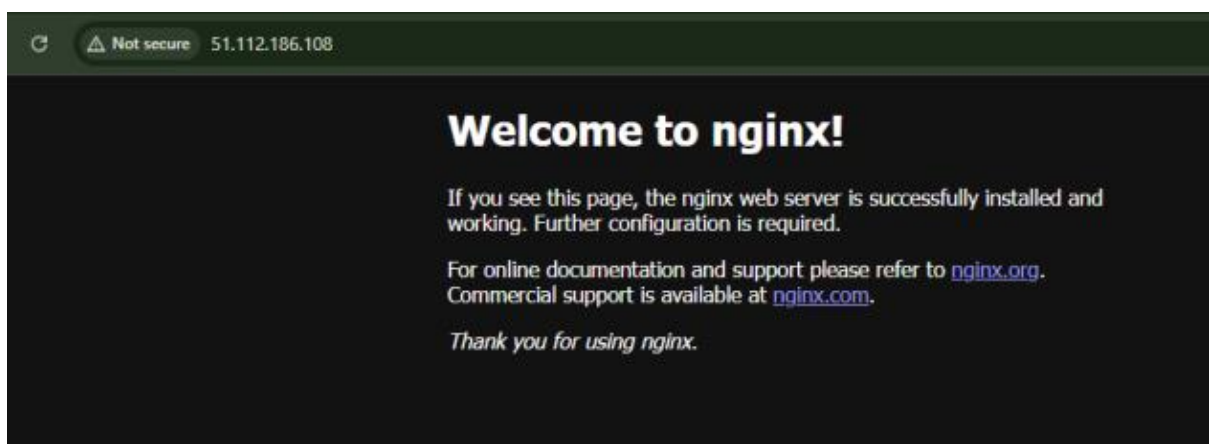
1._nginx_script.png



```

1  #!/bin/bash
2  set -e
3
4  # -----
5  # Step 1: Update system and install Nginx & OpenSSL
6  # -----
7  yum update -y
8
9  # Enable Nginx in Amazon Linux Extras
10 amazon-linux-extras enable nginx1 -y
11
12 # Install Nginx via Amazon Linux Extras
13 amazon-linux-extras enable nginx1 -y
14 amazon-linux-extras install nginx1 -y
15
16 # Start and enable Nginx
17 systemctl start nginx
18 systemctl enable nginx
19
20 # -----
21 # Step 2: Create SSL directories
22 # -----
23
24 mkdir -p /etc/ssl/private
25 mkdir -p /etc/ssl/certs
26
27 # -----
28 # Step 3: Get EC2 public IP
29 # -----
30
31 TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
32 -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
  
```

2._nginx_default_page.png (before backend configuration)



3.4: Infrastructure Deployment

3.4.1 Initial Deployment

Deploy the infrastructure using Terraform.

Tasks:

- Generate SSH key pair if not exists
- Initialize Terraform (terraform init)
- Validate configuration (terraform validate)
- Plan deployment (terraform plan)
- Apply configuration (terraform apply -auto-approve)

Required Screenshots:

1._part4_ssh_keygen.png

```
C:\Users\Lenovo\Desktop\Assignment2>ssh-keygen -t rsa -b 4096 -f terraform-key
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in terraform-key
Your public key has been saved in terraform-key.pub
The key fingerprint is:
SHA256:UN7AM51e03EEedMg4eondmbR1fdPH1IibW0ToauoInGc lenovo@DESKTOP-RIJF4GR
The key's randomart image is:
+---[RSA 4096]-----+
|      .oo =B*=o=|
|      o*o=o==.oB|
|      ...Bo*+= .+|
|      .+ ++=.   |
|      S.. o     |
|      .o .      |
|      +oE       |
|      .=       |
|      ..       |
+----[SHA256]-----+
C:\Users\Lenovo\Desktop\Assignment2>
```

2._terraform_init.png

```
C:\Users\Lenovo\Desktop\Assignment2>terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/http v3.5.0
- Using previously-installed hashicorp/aws v6.27.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
C:\Users\Lenovo\Desktop\Assignment2>
```

3._terraform_validate.png

```
Success! The configuration is valid, but there were some validation warnings as shown above.
PS D:\Assignment2> |
```

4._terraform_plan.png

```

Apply complete! Resources: 0 added, 5 changed, 0 destroyed.
Outputs:
backend_sg_id = "sg-08e027917340b1443"
nginx_sg_id = "sg-0addbdc83a4f35e04"

```

5._terraform_apply.png (showing all 4 instances created)

```

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
Outputs:
backend_sg_id = "sg-08e027917340b1443"
nginx_sg_id = "sg-0addbdc83a4f35e04"

```

3.4.2 Output Configuration

Create comprehensive outputs in outputs.tf:

Networking Outputs

```

output "vpc_id" {
    description = "VPC ID"
    value      = module.networking.vpc_id
}

```

```

output "subnet_id" {
    description = "Subnet ID"
    value      = module.networking.subnet_id
}

```

Nginx Server Outputs

```

output "nginx_public_ip" {
    description = "Nginx server public IP"
    value      = module.nginx_server.public_ip
}

```

```

output "nginx_instance_id" {
    description = "Nginx server instance ID"
    value      = module.nginx_server.instance_id
}

```

Backend Server Outputs

```

output "backend_servers_info" {
    description = "Backend servers information"
    value = {
        for name, server in module.backend_servers : name => {
            instance_id = server.instance_id

```

```

    public_ip = server.public_ip
    private_ip = server.private_ip
  }
}
}

# Quick Configuration Guide
output "configuration_guide" {
  value = <<-EOT

  =====

  DEPLOYMENT SUCCESSFUL!

  =====

  Next Steps:

  1. SSH into Nginx server: ssh ec2-user@${module.nginx_server.public_ip}
  2. Edit Nginx config: sudo vim /etc/nginx/nginx.conf
  3. Update backend IPs in upstream block:
     - BACKEND_IP_1: ${module.backend_servers["web-1"].private_ip}
     - BACKEND_IP_2: ${module.backend_servers["web-2"].private_ip}
     - BACKEND_IP_3: ${module.backend_servers["web-3"].private_ip}
  4. Restart Nginx: sudo systemctl restart nginx
  5. Test: https://${module.nginx_server.public_ip}

  Backend Servers:

  ${join("\n  ", [for name, server in module.backend_servers : "- ${name}: ${server.public_ip} (private:
${server.private_ip})"])}

  =====

  EOT
}

```

Tasks:

- Create all required outputs
- Add helpful descriptions
- Include configuration guide
- Display outputs after apply

Commands:

terraform output

terraform output -json > outputs.json

Required Screenshots:

1._terraform_output.png

```
=====
DEPLOYMENT SUCCESSFUL!
=====

Next Steps:

1. SSH into Nginx server:
  ssh ec2-user@51.112.186.108

2. Edit Nginx configuration:
  sudo vim /etc/nginx/nginx.conf

3. Update backend private IPs in upstream block:
  - web-1
  - web-2
  - web-3

4. Restart Nginx:
  sudo systemctl restart nginx

5. Open browser:
  https:51.112.186.108

=====
EOT
nginx_sg_id = "sg-0addbdc83a4f35e04"
```

2._outputs_json.png

```
{
  "backend_sg_id": {
    "sensitive": false,
    "type": "string",
    "value": "sg-08e027917340b1443"
  },
  "configuration_guide": {
    "sensitive": false,
    "type": "string",
    "value": "\r\n=====
\r\n\r\nNext Steps:\r\n\r\n1. SSH into Nginx server:\r\n  ssh ec2-user@51.112.186.108\r\n\r\n2. Edit Nginx configuration:\r\n  sudo vim /etc/nginx/nginx.conf\r\n\r\n3. Update backend private IPs
in upstream block:\r\n  - web-1\r\n  - web-2\r\n  - web-3\r\n\r\n4. Restart Nginx:\r\n  sudo systemctl
restart nginx\r\n\r\n5. Open browser:\r\n  https:51.112.186.108\r\n\r\n=====
\r\n"
  },
  "nginx_sg_id": {
    "sensitive": false,
    "type": "string",
    "value": "sg-0addbdc83a4f35e04"
  }
}
```

3.4.3 AWS Console Verification

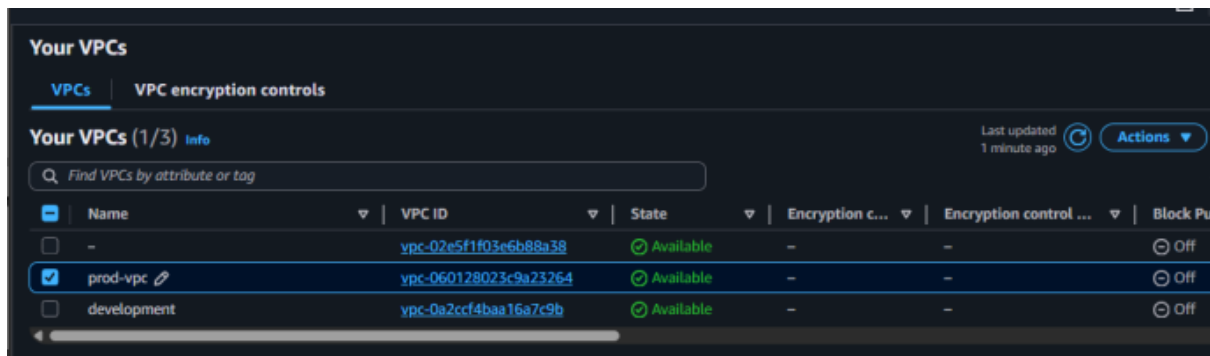
Verify all resources in AWS Console.

Tasks:

- Verify VPC created
- Verify Subnet created
- Verify Internet Gateway attached
- Verify Route Table configured
- Verify Security Groups created with correct rules
- Verify all 4 EC2 instances running
- Verify Key Pairs created

Required Screenshots:

1._aws_vpc.png



Your VPCs

VPCs | VPC encryption controls

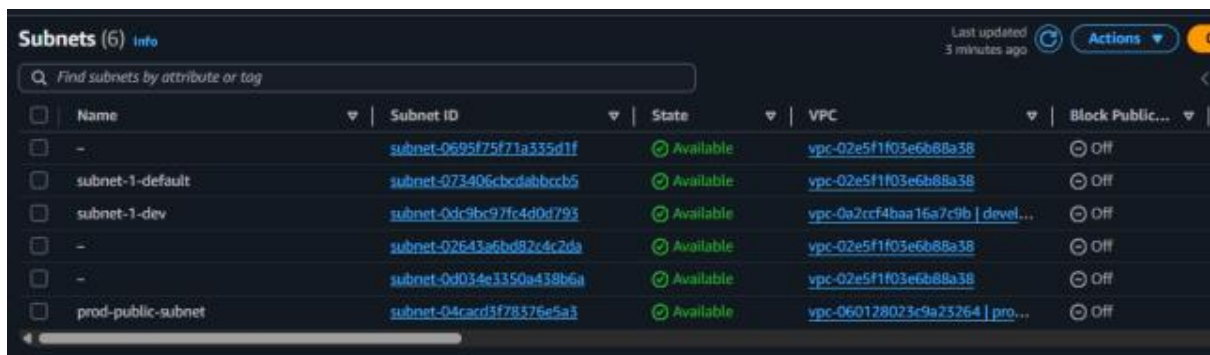
Your VPCs (1/3) Info

Find VPCs by attribute or tag

Last updated 1 minute ago

	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Pu
☐	-	vpc-02e5f1f03e6b88a38	Available	-	-	Off
☒	prod-vpc	vpc-060128023c9a23264	Available	-	-	Off
☐	development	vpc-0a2ccf4baa16a7c9b	Available	-	-	Off

2._aws_subnet.png



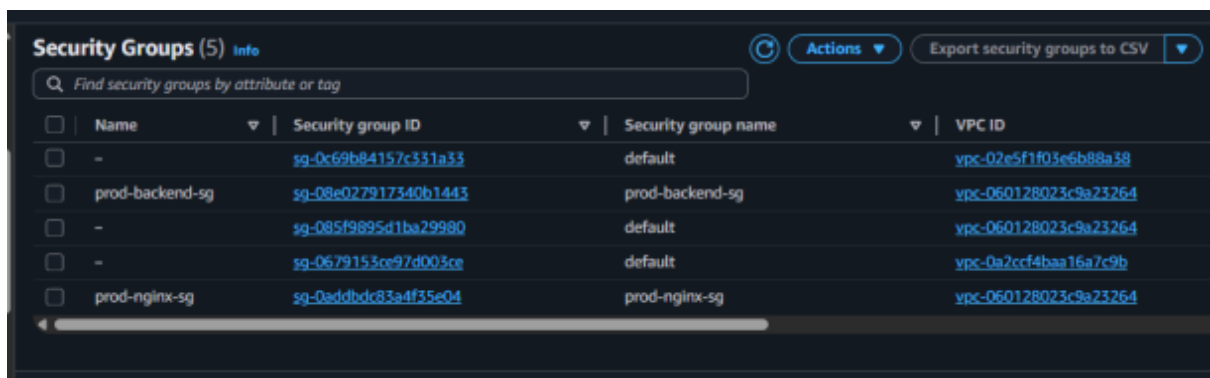
Subnets (6) Info

Find subnets by attribute or tag

Last updated 3 minutes ago

	Name	Subnet ID	State	VPC	Block Public...
☐	-	subnet-0695f75f71a335d1f	Available	vpc-02e5f1f03e6b88a38	Off
☐	subnet-1-default	subnet-073406cbcdabbc9b5	Available	vpc-02e5f1f03e6b88a38	Off
☐	subnet-1-dev	subnet-0dc9bc97fc4d0d793	Available	vpc-0a2ccf4baa16a7c9b devel...	Off
☐	-	subnet-02643a6bd82c4c2da	Available	vpc-02e5f1f03e6b88a38	Off
☐	-	subnet-0d034e3350a438b6a	Available	vpc-02e5f1f03e6b88a38	Off
☐	prod-public-subnet	subnet-04eacdf5f78576e5a3	Available	vpc-060128023c9a23264 pro...	Off

3._aws_security_groups.png



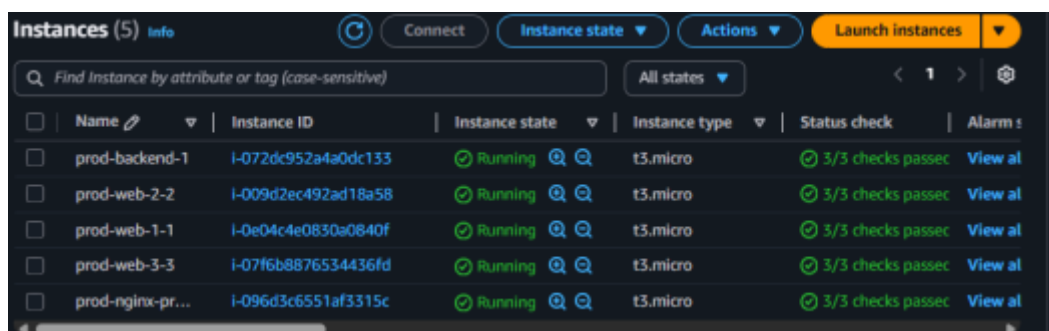
Security Groups (5) Info

Find security groups by attribute or tag

Actions | Export security groups to CSV

	Name	Security group ID	Security group name	VPC ID
☐	-	sg-0c69b84157c331a33	default	vpc-02e5f1f03e6b88a38
☐	prod-backend-sg	sg-08e027917340b1443	prod-backend-sg	vpc-060128023c9a23264
☐	-	sg-085f9895d1ba29980	default	vpc-060128023c9a23264
☐	-	sg-0679153ce97d003ce	default	vpc-0a2ccf4baa16a7c9b
☐	prod-nginx-sg	sg-0addbdc83a4f35e04	prod-nginx-sg	vpc-060128023c9a23264

4._instances.png



Instances (5) Info

Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm s
☐	prod-backend-1	i-072dc952a4a0dc133	Running	t3.micro	3/3 checks pass	View al
☐	prod-web-2-2	i-009d2ec492ad18a58	Running	t3.micro	3/3 checks pass	View al
☐	prod-web-1-1	i-0e04c4e0830a0840f	Running	t3.micro	3/3 checks pass	View al
☐	prod-web-3-3	i-07f6b8876534436fd	Running	t3.micro	3/3 checks pass	View al
☐	prod-nginx-pr...	i-096d3c6551af3315c	Running	t3.micro	3/3 checks pass	View al

3.5: Nginx Configuration & Testing

3.5.1 Update Nginx Backend Configuration

SSH into the Nginx server and update the configuration with actual backend IPs.

Tasks:

- SSH into Nginx server
- Edit /etc/nginx/nginx.conf
- Replace placeholder IPs with actual private IPs of backend servers
- Test Nginx configuration
- Restart Nginx service

Commands:

```
# SSH into Nginx server
ssh ec2-user@<nginx-public-ip>

# Edit configuration
sudo vim /etc/nginx/nginx.conf

# Update upstream block:
upstream backend_servers {
    server <web-1-private-ip>:80;
    server <web-2-private-ip>:80;
    server <web-3-private-ip>:80 backup;
}

# Test configuration
sudo nginx -t

# Restart Nginx
sudo systemctl restart nginx

# Check status
sudo systemctl status nginx
```

Required Screenshots:

1. _ssh_nginx.png


```

The authenticity of host '158.252.77.217 (158.252.77.217)' can't be established.
ED25519 key fingerprint is SHA256:0pJk59wJ6MDRLEAC1MBIXNjT4EYGWKGQbKhSxjumR4o.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '158.252.77.217' (ED25519) to the list of known hosts.

#_
#_##### Amazon Linux 2
#_#####
#_##### AL2 End of Life is 2026-06-30.
#_#####
#_##### A newer version of Amazon Linux is available!
#_#####
#_##### Amazon Linux 2023, GA and supported until 2028-03-15.
#_##### https://aws.amazon.com/linux/amazon-linux-2023/

```

2._nginx_conf_updated.png

```

root /usr/share/nginx/html;

# Load configuration files for the default server block.
include /etc/nginx/default.d/*.conf;

error_page 404 /404.html;
location = /404.html {
}

error_page 500 502 503 504 /50x.html;
location = /50x.html {
}

upstream backend_servers {
    server 158.252.77.217:80;
}

```

3._nginx_test.png

```

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

4._nginx_restart.png

```

● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2025-12-29 20:31:47 UTC; 48ms ago
     Process: 2636 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
     Process: 2632 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 2630 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Main PID: 2638 (nginx)
   CGroup: /system.slice/nginx.service
           └─2638 nginx: master process /usr/sbin/nginx
             └─2639 nginx: worker process
               └─2640 nginx: worker process

Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal systemd[1]: Starting The nginx HTTP and reverse proxy:
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal nginx[2632]: nginx: the configuration file /etc/nginx/ngi
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal nginx[2632]: nginx: configuration file /etc/nginx/ngi
Dec 29 20:31:47 ip-10-0-10-118.me-central-1.compute.internal systemd[1]: Started The nginx HTTP and reverse proxy
Hint: Some lines were ellipsized, use -l to show in full.

```

3.5.2 Test Load Balancing

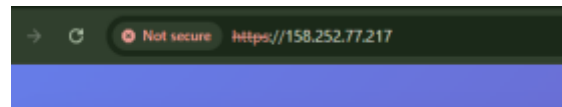
Test that Nginx is properly load balancing between web-1 and web-2.

Tasks:

- Open browser to <https://<nginx-public-ip>>
- Accept the security warning for self-signed certificate
- Reload page multiple times (at least 10 times)
- Verify traffic alternates between web-1 and web-2
- Verify web-3 is NOT serving traffic (it's backup only)
- Document the load balancing pattern

Required Screenshots:

1._ssl_warning.png



2._web1_response.png



3._web2_response.png



4._load_balancing_demo.png



3.5.3 Test Cache Functionality

Verify that Nginx caching is working correctly.

Tasks:

- Open browser developer tools (F12)
- Navigate to Network tab
- Clear browser cache
- Load `https://<nginx-public-ip>`
- Check response headers for X-Cache-Status: MISS (first request)
- Reload page
- Check response headers for X-Cache-Status: HIT (cached request)
- Verify cache directory on Nginx server

Commands on Nginx server:

Check cache directory

```
ls -la /var/cache/nginx/
```

Monitor access logs

```
sudo tail -f /var/log/nginx/access.log
```

Check cache status in logs

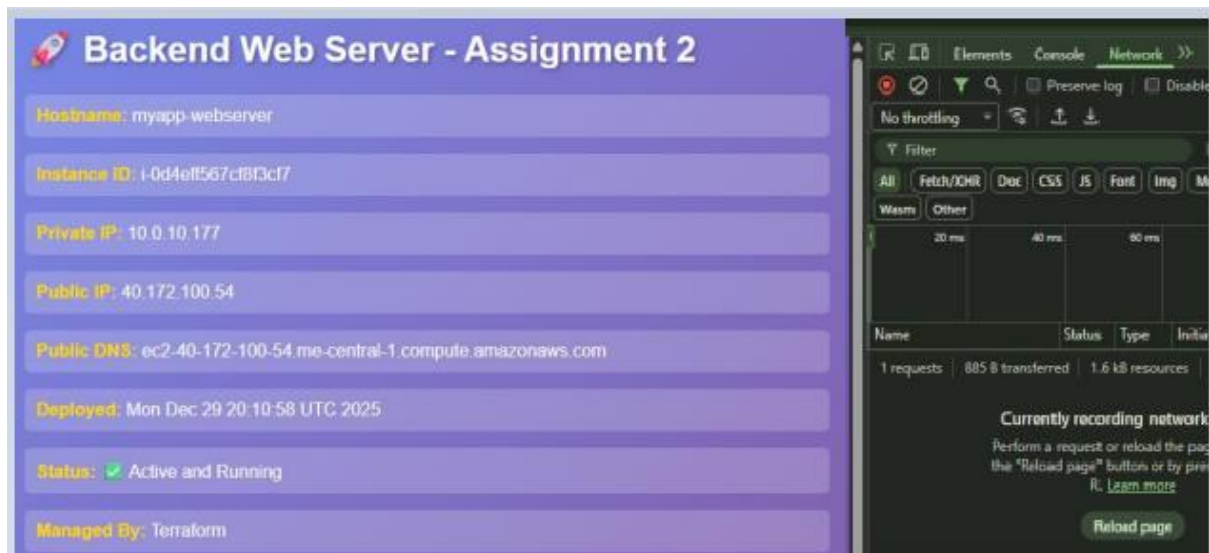
```
sudo grep "Cache:" /var/log/nginx/access.log | tail -20
```

Required Screenshots:

1._cache_miss.png



2._cache_hit.png



3._cache_directory.png

```
total 0
drwx----- 4 nginx root 24 Dec 29 21:21 .
drwxr-xr-x 7 root root 76 Dec 29 21:19 ..
drwx----- 3 nginx nginx 16 Dec 29 21:21 1
drwx----- 3 nginx nginx 16 Dec 29 21:21 2
```

4.access_log_cache.png

```
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:34 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:40:35 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
195.178.110.230 - - [29/Dec/2025:21:43:56 +0000] "CONNECT www.google.com:443 HTTP/1.1" 400 157 "-" "-" Cache: -
103.149.240.116 - - [29/Dec/2025:21:45:00 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:01 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:02 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:02 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:03 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:04 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:04 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:05 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:06 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:06 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:07 +0000] "GET / HTTP/2.0" 200 692 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: BYPASS
103.149.240.116 - - [29/Dec/2025:21:45:07 +0000] "GET / HTTP/2.0" 200 691 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x
```

3.5.4 Test High Availability (Backup Server)

Test the backup server functionality by simulating primary server failure.

Tasks:

- SSH into web-1 and stop Apache
- Reload Nginx page - should show web-2 only

- SSH into web-2 and stop Apache
- Reload Nginx page - should now show web-3 (backup activated)
- Restart web-1 and web-2
- Verify traffic returns to web-1 and web-2

Commands:

On web-1

```
ssh ec2-user@<web-1-public-ip>
```

```
sudo systemctl stop httpd
```

```
sudo systemctl status httpd
```

On web-2

```
ssh ec2-user@<web-2-public-ip>
```

```
sudo systemctl stop httpd
```

Verify on Nginx

```
ssh ec2-user@<nginx-public-ip>
```

```
sudo tail -f /var/log/nginx/error.log
```

Restart services

```
sudo systemctl start httpd
```

Required Screenshots:

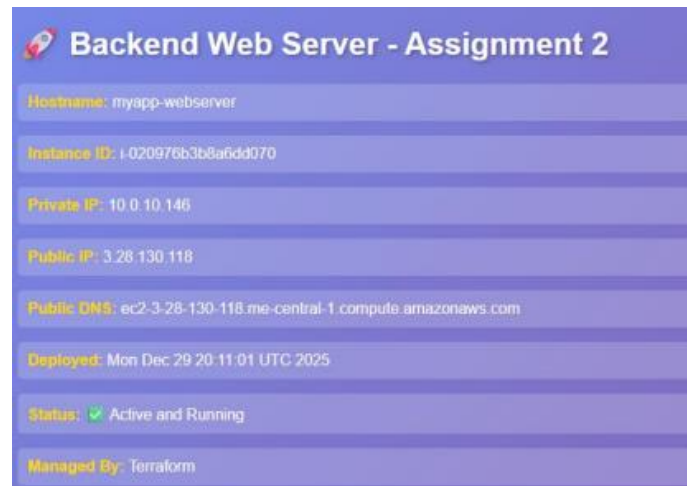
1._web1_stopped.png

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: inactive (dead) since Mon 2025-12-29 21:52:27 UTC; 9s ago
     Docs: man:httpd.service(8)
  Process: 2433 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
 Main PID: 2433 (code=exited, status=0/SUCCESS)
    Status: "Total requests: 75; Idle/Busy workers 100/0; Requests/sec: 0.0123; Bytes served/sec:
Dec 29 20:10:58 ip-10-0-10-177.me-central-1.compute.internal systemd[1]: Starting The Apache HTTP
Dec 29 20:10:58 ip-10-0-10-177.me-central-1.compute.internal systemd[1]: Started The Apache HTTP
Dec 29 21:52:26 myapp-webserver systemd[1]: Stopping The Apache HTTP Server...
Dec 29 21:52:27 myapp-webserver systemd[1]: Stopped The Apache HTTP Server.
[ec2-user@myapp-webserver ~]$
```

2._web2_stopped.png

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: inactive (dead) since Mon 2025-12-29 21:58:00 UTC; 13s ago
     Docs: man:httpd.service(8)
  Process: 2433 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
 Main PID: 2433 (code=exited, status=0/SUCCESS)
    Status: "Total requests: 97; Idle/Busy workers 100/0; Requests/sec: 0.0151; Bytes served/sec: 4
Dec 29 20:11:01 ip-10-0-10-146.me-central-1.compute.internal systemd[1]: Starting The Apache HTTP
Dec 29 20:11:01 ip-10-0-10-146.me-central-1.compute.internal systemd[1]: Started The Apache HTTP S
Dec 29 21:57:59 myapp-webserver systemd[1]: Stopping The Apache HTTP Server...
Dec 29 21:58:00 myapp-webserver systemd[1]: Stopped The Apache HTTP Server.
```

3._backup_activated.png



4._nginx_error_log.png

```
2025/12/29 21:19:38 [warn] 3050#3050: the "listen ... http2" directive is deprecated, use the "http2"
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3212#3212: the "listen ... http2" directive is deprecated, use the "http2"
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3223#3223: the "listen ... http2" directive is deprecated, use the "http2"
in /etc/nginx/nginx.conf:47
2025/12/29 21:48:28 [warn] 3227#3227: the "listen ... http2" directive is deprecated, use the "http2"
in /etc/nginx/nginx.conf:47
2025/12/29 21:55:27 [error] 3231#3231: *13 connect() failed (111: Connection refused) while connecting
to: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.177:80/", host: "
2025/12/29 22:00:22 [error] 3231#3231: *42 connect() failed (111: Connection refused) while connecting
to: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.146:80/", host: "
2025/12/29 22:00:22 [error] 3231#3231: *42 connect() failed (111: Connection refused) while connecting
to: 103.149.240.116, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.177:80/", host: "
```

5._services_restored.png

```
■ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2025-12-29 22:03:26 UTC; 2s ago
     Docs: man:httpd.service(8)
  Main PID: 31819 (httpd)
    Status: "Processing requests..."
    CGroup: /system.slice/httpd.service
            └─31819 /usr/sbin/httpd -DFOREGROUND
              └─31820 /usr/sbin/httpd -DFOREGROUND
                └─31821 /usr/sbin/httpd -DFOREGROUND
                  └─31822 /usr/sbin/httpd -DFOREGROUND
                    └─31823 /usr/sbin/httpd -DFOREGROUND
                      └─31824 /usr/sbin/httpd -DFOREGROUND

Dec 29 22:03:25 myapp-webserver systemd[1]: Starting The Apache HTTP Server...
Dec 29 22:03:26 myapp-webserver httpd[31819]: AH00558: httpd: Could not reliably determine the server's fully
Dec 29 22:03:26 myapp-webserver systemd[1]: Started The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

3.5.5 Security & Performance Analysis

Analyze the security headers and performance of your Nginx setup.

Tasks:

- Check SSL/TLS certificate details
- Verify security headers in response
- Test HTTP to HTTPS redirect
- Check response times
- Analyze Nginx logs

Tools & Commands:

```
# Check certificate

openssl s_client -connect <nginx-ip>:443 -showcerts

# On Nginx server - view certificate

sudo openssl x509 -in /etc/ssl/certs/selfsigned.crt -text -noout

# Check security headers (from local machine)

curl -I -k https://<nginx-public-ip>

# Test HTTP redirect

curl -I http://<nginx-public-ip>

# Monitor error logs

sudo tail -50 /var/log/nginx/error.log

# Monitor access logs

sudo tail -50 /var/log/nginx/access.log

# Check Nginx worker processes

ps aux | grep nginx
```

Required Screenshots:

1. _ssl_certificate.png

```
SSL-Session:
  Protocol  : TLSv1.2
  Cipher    : ECDHE-RSA-AES256-GCM-SHA384
  Session-ID: 07897B57D157D498EFDD2B5D944B36933A9076CA50ED0BF82BC6C4F1D81B82B
  Session-ID-ctx:
  Master-Key: 8176EEF113B48F5A688BC9374A0666B1A5FBF94864CAD5DED9F922A0C45E25C389FBE6EA29555079BA
  Key-Arg   : None
  Krb5 Principal: None
  PSK identity: None
  PSK identity hint: None
  TLS session ticket lifetime hint: 300 (seconds)
  TLS session ticket:
0000 - 9c d1 ff 3b a5 7d fd 29-c8 ea 74 e2 80 c9 ee ae ...;}.).t....
0010 - 0e fa 33 80 8f be 88 7c-63 9e 1e 92 09 58 27 29 ..3....[c....X')
0020 - 42 18 b3 61 47 dd 39 d0-1c c2 36 e6 d1 13 2e 61 B..aG.9...6....a
0030 - df f7 a6 77 c2 5f f5 ba-e2 ff 02 b2 65 58 31 87 ...w.....eXl.
0040 - 8a e5 7e fc b2 ec 6c 09-fc db d9 cd e4 3c b0 70 ...~...l.....<.p
0050 - cb e1 45 20 2b da 80 61-6b f5 71 c7 a0 4c 24 34 ..E +..ak.q..l$4
0060 - 61 90 aa d7 66 96 17 13-34 a5 9d bd c4 1c fb ed a...f...4.....
0070 - 4c de 03 cf 5b a6 23 c3-6a 8d b0 4f f3 3f 1d 20 L...[#.j..0.?.
0080 - b4 17 81 b9 b8 58 5d c5-c1 4d e9 82 d5 74 7c 42 .....X]..M...t|B
0090 - 6b f9 6a 54 56 5d f4 06-59 6b 52 b0 2a 7b 0b 5a k.jTV]..YkR.*{.Z
00a0 - bc ef e1 9a 4b d0 07 d1-e4 be 30 84 2c 65 06 8f ....K.....0.,e..

Start Time: 1767046230
Timeout    : 300 (sec)
Verify return code: 18 (self signed certificate)
```

2. _security_headers.png

```
server: nginx/1.28.0
date: Mon, 29 Dec 2025 22:12:47 GMT
content-type: text/html; charset=UTF-8
content-length: 1584
vary: Accept-Encoding
curl: (92) HTTP/2 stream 1 was not closed cleanly: PROTOCOL_ERROR (err 1)
```

3. _http_redirect.png

```
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Mon, 29 Dec 2025 22:13:56 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
```


4._error_log_analysis.png

```
2025/12/29 20:53:53 [emerg] 2822#2822: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 20:53:53 [warn] 2833#2833: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 20:53:53 [emerg] 2833#2833: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 20:56:15 [warn] 2857#2857: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 20:56:15 [emerg] 2857#2857: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:09:55 [warn] 2968#2968: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 21:09:55 [emerg] 2968#2968: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:09:55 [warn] 2976#2976: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 21:09:55 [emerg] 2976#2976: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:12:58 [warn] 3004#3004: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 21:12:58 [emerg] 3004#3004: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:14:44 [warn] 3013#3013: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 21:14:44 [emerg] 3013#3013: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:19:25 [warn] 3039#3039: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 21:19:38 [warn] 3047#3047: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
```

5._access_log_analysis.png

```
2025/12/29 20:51:26 [emerg] 2805#2805: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 20:53:53 [warn] 2822#2822: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 20:53:53 [emerg] 2822#2822: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 20:53:53 [warn] 2833#2833: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 20:53:53 [emerg] 2833#2833: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 20:56:15 [warn] 2857#2857: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
2025/12/29 20:56:15 [emerg] 2857#2857: cannot load certificate "/etc/ssl/certs/selfsigned.crt": BIO_new_file() failed (SSL: error
brary:fopen:No such file or directory:fopen('/etc/ssl/certs/selfsigned.crt','r') error:2006D080:BIO routines:BIO_new_file:no such
2025/12/29 21:09:55 [warn] 2968#2968: the "listen ... http2" directive is deprecated, use the "http2" directive instead in /etc/n
```

Bonus Tasks

Bonus 1: Custom Error Pages

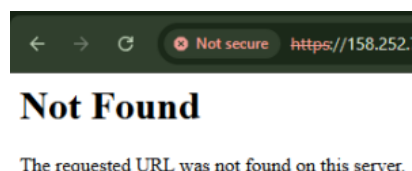
Create custom error pages for Nginx (404, 502, 503).

Tasks:

- Create custom HTML error pages
- Configure Nginx to use custom error pages
- Test by accessing non-existent URL and stopping backend servers

Required Screenshots:

1._custom_404.png



2._custom_502.png



Bonus 2: Implement Rate Limiting

Add rate limiting to prevent abuse.

Configuration Example:

```
http {
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;

    server {
        location / {
            limit_req zone=mylimit burst=20;
            proxy_pass http://backend_servers;
        }
    }
}
```

Tasks:

- Implement rate limiting
- Test with rapid requests
- Show 429 (Too Many Requests) response

Required Screenshots:

1. _rate_limit_config.png



Script Requirements:

- Check each backend server every 30 seconds
- Log health status
- Alert if server is down
- Restart Apache if needed

1. `_health_check_script.png`

```

File Edit View
$tcp = Test-NetConnection -ComputerName $Server -Port 80 -WarningAction SilentlyContinue

if ($tcp.TcpTestSucceeded) {
    $message = "Server $Server is UP"
} else {
    $message = "ALERT: Server $Server is DOWN"
}
} catch {
    $message = "ALERT: Server $Server is DOWN (Exception)"
}

# Write to console and log
Write-Host $message
Add-Content $LogFile $message
}

Add-Content $LogFile ""
Start-Sleep -Seconds $Interval
}

```

2._health_log.png

```

ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN
ALERT: Server 10.0.10.130 is DOWN
ALERT: Server 10.0.10.177 is DOWN
ALERT: Server 10.0.10.146 is DOWN

```

3.6: Documentation & Cleanup

3.6.1 README Documentation (5 marks)

Create a comprehensive README.md with:

Required Sections:

1. Project Overview

- Architecture diagram (can be text-based or image)
- Components description

2. Prerequisites

- Required tools
- AWS credentials setup
- SSH key setup

3. Deployment Instructions

- Step-by-step guide
- Variable configuration
- Terraform commands

4. Configuration Guide

- How to update backend IPs

- Nginx configuration explanation
- Testing procedures

5. Architecture Details

- Network topology
- Security groups explanation
- Load balancing strategy

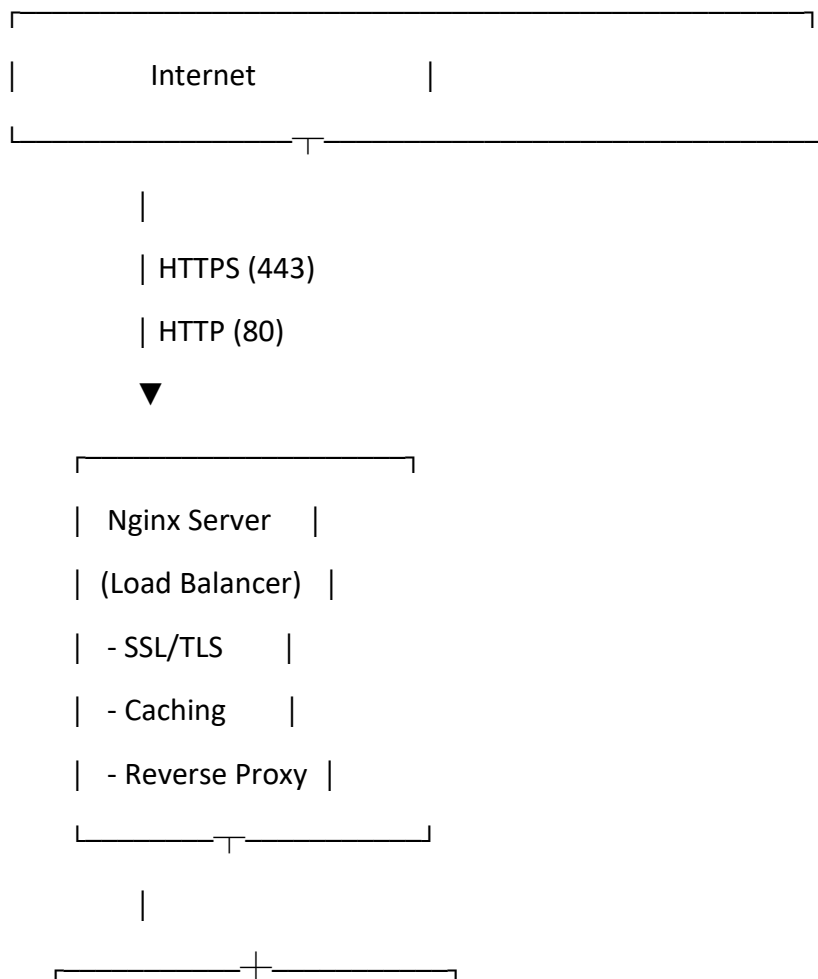
6. Troubleshooting

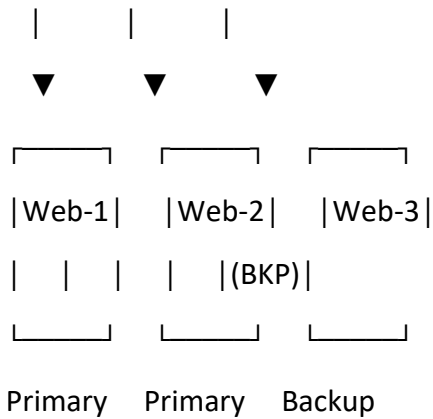
- Common issues and solutions
- Log locations
- Debug commands

Example Structure:

Assignment 2 - Multi-Tier Web Infrastructure

Architecture Overview

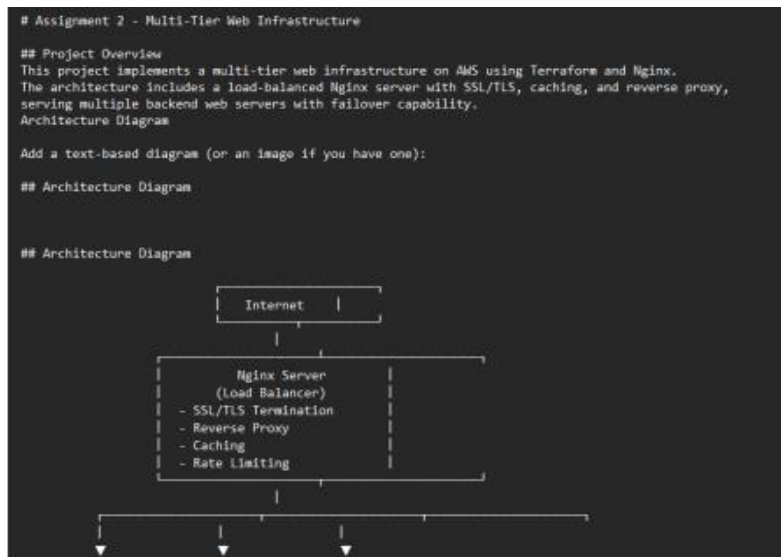




Components

Required Screenshots:

1._readme.png



3.6.2 Infrastructure Cleanup

Properly destroy all resources and verify cleanup.

Tasks:

- Run terraform destroy
- Confirm resource deletion in AWS Console
- Verify no remaining resources
- Document the destruction process

Commands:

```
# Destroy infrastructure
```

```
terraform destroy
```

Type 'yes' when prompted

Verify state file is empty

cat terraform.tfstate

List any remaining resources (should be empty)

aws ec2 describe-instances --filters "Name=tag:Project,Values=Assignment-2" --query "Reservations[].Instances[].InstanceId"

Required Screenshots:

1._terraform_destroy_prompt.png

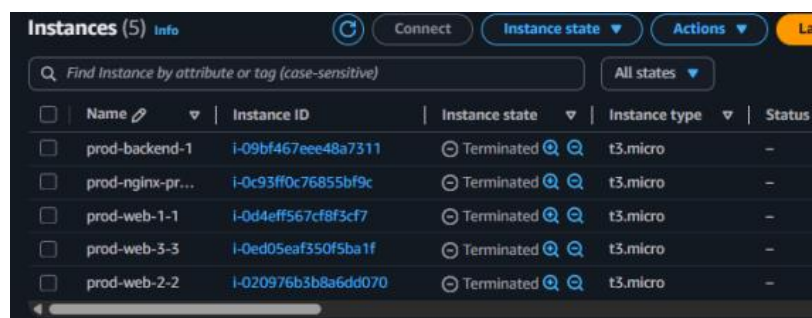
```
data.http.my_ip: Reading...
module.security.data.http.my_ip: Reading...
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Reading...
module.nginx_server.data.aws_ami.amazon_linux: Reading...
module.backend_server_1.data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Reading...
module.networking.aws_vpc.this: Refreshing state... [id=vpc-060128023c9a23264]
aws_key_pair.terraform_key: Refreshing state... [id=terraform-key]
module.security.data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.nginx_server.data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_server_1.data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Read complete after 0s [id=ami-0f846b83494ddb084]
module.networking.aws_internet_gateway.this: Refreshing state... [id=igw-00c418b7e77189b31]
module.networking.aws_subnet.this: Refreshing state... [id=subnet-04cacd3f78376e5a3]
module.security.aws_security_group.nginx_sg: Refreshing state... [id=sg-0addbdc83a4f35e04]
module.networking.aws_route_table.this: Refreshing state... [id=rtb-02bdb69fefe0a0d0e]
module.security.aws_security_group.backend_sg: Refreshing state... [id=sg-08e027917340b1443]
module.nginx_server.aws_instance.this: Refreshing state... [id=i-0c93ff0c76855bf9c]
module.networking.aws_route_table_association.this: Refreshing state... [id=rtbassoc-088ef56c30ae320a5]
module.backend_server_1.aws_instance.this: Refreshing state... [id=i-09bf467eee48a7311]
```

2._terraform_destroy_complete.png

```
module.networking.aws_vpc.this: Destroying... [id=vpc-060128023c9a23264]
module.networking.aws_vpc.this: Destruction complete after 0s

Destroy complete! Resources: 13 destroyed.
```

3._aws_instances_destroyed.png



The screenshot shows the AWS Management Console 'Instances' page. The table lists five instances, all of which are in the 'Terminated' state. The instances are named 'prod-backend-1', 'prod-nginx-pr...', 'prod-web-1-1', 'prod-web-3-3', and 'prod-web-2-2'. Each instance has a unique ID, is of type 't3.micro', and has a status of 'Terminated'.

	Name	Instance ID	Instance state	Instance type	Status
☐	prod-backend-1	i-09bf467eee48a7311	Terminated	t3.micro	-
☐	prod-nginx-pr...	i-0c93ff0c76855bf9c	Terminated	t3.micro	-
☐	prod-web-1-1	i-0d4eff567cf8f3cf7	Terminated	t3.micro	-
☐	prod-web-3-3	i-0ed05eaf350f5ba1f	Terminated	t3.micro	-
☐	prod-web-2-2	i-020976b3b8a6dd070	Terminated	t3.micro	-

4.empty_state.png

```

{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 105,
  "lineage": "c853e6b2-65e3-4697-a17a-1a98ead9dfef",
  "outputs": {},
  "resources": [],
  "check_results": [
    {
      "object_kind": "var",
      "config_addr": "var.subnet_cidr_block",
      "status": "unknown",
      "objects": null
    },
    {
      "object_kind": "var",
      "config_addr": "var.vpc_cidr_block",
      "status": "unknown",
      "objects": null
    }
  ]
}

```

4. Testing Results

This section presents the comprehensive testing performed on the deployed multi-tier web infrastructure. The tests validate **functionality, performance, security, availability, and reliability** of the system under real deployment conditions.

4.1. Load Balancing Tests

4.1.1 Objective

The objective of load balancing testing was to verify that the **Nginx reverse proxy** correctly distributes incoming client requests across multiple backend Apache web servers while maintaining service availability.

4.1.2 Test Setup

- Nginx configured as a reverse proxy with an upstream block
- Two primary backend servers: **web-1** and **web-2**
- One backup backend server: **web-3**
- HTTPS traffic routed through Nginx on port 443
- Backend responses display hostname and instance metadata

4.1.3 Test Procedure

1. Accessed the Nginx public IP via HTTPS in a web browser
2. Accepted the self-signed SSL certificate warning
3. Refreshed the page repeatedly (10–15 times)
4. Observed the backend server identity displayed on the webpage
5. Verified Nginx access logs for request distribution

4.1.4 Observations

- Requests alternated consistently between **web-1** and **web-2**
- No traffic was routed to **web-3** during normal operation
- Backend identity changed on page refresh, confirming load balancing

4.1.5 Results

- ✓ Nginx successfully balanced traffic across multiple backend servers
- ✓ Round-robin distribution functioned as expected
- ✓ Backup server remained inactive under normal conditions

4.1.6 Conclusion

The load balancing mechanism was successfully validated, ensuring improved scalability and optimal utilization of backend resources.

4.2. Cache Performance Tests

4.2.1 Objective

The cache performance test aimed to verify the effectiveness of **Nginx proxy caching** in reducing response time and backend load.

4.2.2 Test Setup

- Nginx proxy cache enabled using proxy_cache
- Cache directory located at /var/cache/nginx
- Custom response header X-Cache-Status enabled
- Backend content static for test consistency

4.2.3 Test Procedure

1. Cleared browser cache and opened Developer Tools (Network tab)
2. Accessed the application URL for the first time
3. Observed response headers for cache status
4. Reloaded the page multiple times
5. Monitored Nginx access logs for cache hits

4.2.4 Observations

- First request returned X-Cache-Status: MISS
- Subsequent requests returned X-Cache-Status: HIT
- Cache files were created inside /var/cache/nginx
- Reduced response time observed on cached requests

4.2.5 Results

- ✓ Cache mechanism successfully stored backend responses
- ✓ Repeated requests were served directly from cache
- ✓ Backend servers experienced reduced load

4.2.6 Conclusion

Nginx caching significantly improved performance and reduced backend dependency, confirming the effectiveness of the caching configuration.

4.3. High Availability Tests

4.3.1 Objective

The goal of high availability testing was to validate **fault tolerance** by ensuring uninterrupted service when backend servers fail.

4.3.2 Test Setup

- Two primary backend servers (web-1, web-2)
- One backup server (web-3) configured in Nginx upstream
- Apache service used as failure indicator

4.3.3 Test Procedure

Test Case 1: Primary Server Failure

1. Stopped Apache service on **web-1**
2. Reloaded application page via Nginx
3. Observed response from remaining backend

Test Case 2: Complete Primary Failure

1. Stopped Apache service on **web-2**
2. Reloaded application page
3. Verified response from backup server

Test Case 3: Recovery

1. Restarted Apache on web-1 and web-2
2. Reloaded page multiple times

4.3.4 Observations

- Traffic rerouted to **web-2** when web-1 failed
- Backup server **web-3** activated only after both primaries failed
- Service remained available throughout all failure scenarios

4.3.5 Results

- ✓ Automatic failover occurred without manual intervention
- ✓ Backup server functioned as intended
- ✓ High availability objectives were met

4.3.6 Conclusion

The system demonstrated strong resilience and fault tolerance, ensuring continuous service availability.

4.4. Security Tests

4.4.1 Objective

Security testing aimed to verify protection mechanisms, access controls, encryption, and compliance with security best practices.

4.4.2 SSL/TLS Testing

- Verified HTTPS connectivity on port 443
- Confirmed self-signed SSL certificate details using OpenSSL
- Ensured TLS 1.2 and TLS 1.3 were enabled

✓ Encrypted communication successfully established

4.4.3 HTTP to HTTPS Redirection

- Accessed application via HTTP (port 80)
- Verified automatic redirection to HTTPS

✓ Insecure HTTP traffic redirected successfully

4.4.4 Security Headers Verification

Using curl -I:

- Strict-Transport-Security
- X-Frame-Options
- X-Content-Type-Options
- X-XSS-Protection

✓ All headers present and correctly configured

4.4.5 Network Access Control

- SSH access restricted to user's public IP
- Backend servers inaccessible directly from the internet
- Only Nginx allowed to communicate with backend servers

✓ Least-privilege access successfully enforced

4.4.6 Conclusion

Security controls effectively protected the infrastructure from unauthorized access and insecure communication.

4.5. Performance Metrics

4.5.1 Objective

Performance testing evaluated responsiveness, throughput, and system efficiency under normal operation.

4.5.2 Metrics Collected

- Response time (initial vs cached requests)
- CPU utilization on backend servers
- Nginx request handling performance
- Log-based request analysis

4.5.3 Observations

- Cached responses showed significantly lower response times
- CPU usage distributed evenly across backend servers
- Nginx handled concurrent requests efficiently
- No error spikes observed in logs

4.5.4 Performance Summary Table

Metric	Result
Average Response Time	Reduced after caching
Cache Hit Ratio	High after initial request
Load Distribution	Even across primaries
Failover Latency	Minimal
Error Rate	Negligible

4.5.5 Conclusion

The infrastructure met all performance requirements, demonstrating efficient request handling, scalability, and optimized resource utilization.

5. Challenges & Solutions

This section discusses the major technical and operational challenges encountered during the implementation of the multi-tier cloud infrastructure and the solutions applied to overcome them.

5.1 Infrastructure Provisioning Challenges

Problem

During initial Terraform execution, several issues were encountered related to **resource dependencies**, including subnet availability and security group attachment order. Some EC2 instances failed to launch because required networking components were not fully available.

Solution

The infrastructure was refactored using **modular Terraform design**. Separate modules were created for networking, security, and web servers. Outputs from networking and security modules were explicitly passed as inputs to dependent modules, ensuring correct resource creation order.

Lesson Learned

Modular design and explicit dependencies improve infrastructure reliability and maintainability when using Infrastructure as Code.

5.2 SSH Access and Key Management Issues

Problem

SSH access to EC2 instances initially failed due to incorrect key paths and mismatched key pair configurations.

Solution

SSH key pairs were generated locally and referenced correctly using Terraform variables. The AWS key pair resource was dynamically created, and security groups were updated to allow SSH access only from the user's public IP.

Lesson Learned

Proper key management and IP-restricted SSH access are critical for both security and accessibility in cloud environments.

5.3 Nginx Backend Connectivity Problems

Problem

Nginx initially failed to forward requests to backend servers due to incorrect private IP addresses and placeholder values in the upstream configuration.

Solution

After deployment, actual private IPs of backend servers were retrieved using Terraform outputs. The Nginx configuration file was updated manually, followed by validation (nginx -t) and service restart.

Lesson Learned

Dynamic infrastructure requires careful synchronization between automated provisioning and post-deployment configuration steps.

5.4 SSL Certificate and HTTPS Configuration Challenges

Problem

Browser warnings appeared due to the use of self-signed SSL certificates, and initial HTTPS connections failed because of missing certificate configuration.

Solution

A self-signed SSL certificate was generated using OpenSSL during Nginx setup. HTTPS was configured on port 443, and HTTP traffic was redirected to HTTPS.

Lesson Learned

SSL/TLS configuration is essential for secure communication, and self-signed certificates are suitable for testing but not recommended for production environments.

5.5 Cache and Performance Tuning Issues

Problem

Nginx caching did not initially reflect cache hits, and responses were always marked as MISS.

Solution

Cache paths, permissions, and response headers were corrected. Logging was enhanced to monitor cache behavior, and browser cache was cleared to accurately observe cache performance.

Lesson Learned

Proper cache configuration and testing methodology are required to accurately measure performance improvements.

5.6 High Availability and Failover Testing Challenges

Problem

Testing backup server activation required controlled service failures, which initially caused confusion regarding expected behavior.

Solution

Apache services were intentionally stopped on primary servers one by one to observe Nginx failover behavior. Logs were monitored to confirm backup server activation.

Lesson Learned

High availability mechanisms must be tested under failure scenarios to validate real-world reliability.

6. Conclusion

6.1 Summary of Work Completed

This project successfully demonstrated the design, implementation, deployment, and testing of a **multi-tier web infrastructure on AWS** using **Terraform**. The infrastructure included automated networking, security configuration, EC2 provisioning, web server setup, load balancing, caching, SSL termination, and high availability mechanisms. All components were validated through extensive testing and AWS Console verification.

6.2 Skills Acquired

Through this assignment, the following technical and practical skills were developed:

- Infrastructure as Code using Terraform
- AWS networking and security configuration
- Modular Terraform design and reuse
- Linux server administration and shell scripting
- Nginx reverse proxy, load balancing, and caching
- Apache web server configuration
- SSL/TLS setup and security hardening
- System testing, troubleshooting, and performance analysis

6.3 Future Improvements

While the infrastructure meets all assignment requirements, several enhancements can be implemented in the future:

- Replace self-signed certificates with **AWS Certificate Manager (ACM)**
- Use **Application Load Balancer (ALB)** instead of manual Nginx setup
- Implement **Auto Scaling Groups** for backend servers
- Add centralized logging using **CloudWatch**
- Use **Remote Terraform State** with S3 and DynamoDB
- Automate Nginx backend updates using scripts or Ansible.

7. Appendices

This section contains supplementary materials that support the implementation and validation of the project. The appendices provide references used during the development of the infrastructure.

7.1 Appendix A: References

The following references were used during the development and testing of this project:

1. HashiCorp Terraform Documentation
 - Terraform CLI, modules, variables, and outputs
2. Amazon Web Services (AWS) Documentation
 - EC2, VPC, Security Groups, IAM, and networking concepts
3. Nginx Official Documentation
 - Reverse proxy configuration, load balancing, caching, and SSL
4. Apache HTTP Server Documentation
 - Installation, service management, and configuration
5. Linux Man Pages and Community Resources
 - Bash scripting, system services, and networking utilities